

激光颗粒物监测系统入门手册

这份文档是给小白看的。目标不是一上来把你训练成大气激光雷达专家，而是先让你真正看懂：这套系统到底在测什么、为什么能测、公式每一项是什么意思、工程上应该怎么做、软件界面应该长什么样。这份文档聚焦的是颗粒物和扬尘，也就是 PM2.5、PM10、施工扬尘、道路积尘再悬浮、烟羽、粉尘团这类目标。它不是一份“什么气体都能测”的大全，更不是一份只会堆公式的科研摘要。

0. 先用一句人话讲清楚这套系统

你可以把它理解成一套“会发光的空气扫描仪”：

1. 它向空气里打一束很窄的激光。
2. 空气里的分子、灰尘、烟雾、雾滴会把一小部分光散射回来。
3. 仪器接收这些返回光。
4. 因为不同距离的回波到达时间不同，所以可以知道“哪一段空间里有多少回波”。
5. 再通过算法把“回波强弱”变成“颗粒物分布、污染层位置、浓度热点、三维污染云图”。

如果你过去只接触过普通激光雷达或三维点云，这里最重要的区别是：

- 普通 3D LiDAR 主要看固体物体表面。
- 大气激光雷达主要看空气本身对光的散射和衰减。
- 它得到的原始结果不是建筑物轮廓，而是一条沿距离变化的回波曲线，或者一张时间-高度热力图。

1. 这套系统到底能做什么，不能做什么

1.1 它能做什么

如果目标是工地扬尘、道路扬尘、料场粉尘、烟羽、雾霾层，这套系统非常合适。它擅长：

1. 发现哪里有颗粒物或粉尘团。
2. 判断粉尘层大概在哪个高度、哪个方向、离设备多远。
3. 连续扫描，做出二维剖面 and 三维空间分布图。
4. 结合地面 PM 传感器、湿度和气象数据，估算 PM2.5 / PM10 浓度。
5. 提取污染热点的三维坐标，联动喷雾炮或洒水车做闭环控制。

1.2 它不能直接做到什么

作为小白，最容易误解的就是“既然都叫激光雷达，是不是打一束光就能直接读出 PM2.5 数值”。答案是不行。

它不能直接做到：

1. 不经过标定就直接输出绝对准确的 PM2.5、PM10 质量浓度。
2. 用一台设备同时准确测完所有气体污染物。
3. 像摄像头一样直接拍到“污染物真实外形”。

4. 不做安全设计就拿到户外随便打激光。

1.3 为什么它不能直接“称重” PM2.5

因为它本质上测到的是光学量，而不是颗粒物的实际重量。

它首先测到的是：

- 后向散射强不强。
- 激光在空气里被削弱得厉不厉害。
- 哪些高度、哪些方向回波更强。

真正的 PM2.5 / PM10 质量浓度，通常要通过这些光学量再去和下面这些信息一起标定：

- 地面 PM 参考仪。
- 相对湿度 RH。
- 温度、气压。
- 颗粒物类型。
- 本地季节和源类型。

所以更准确的人话是：

激光雷达先测“光学浓度特征”，再通过本地校准模型去估算“质量浓度”。

2. 先把几个最关键的词看懂

下面这些词后面会反复出现。先有一个直觉，后面看公式就轻松很多。

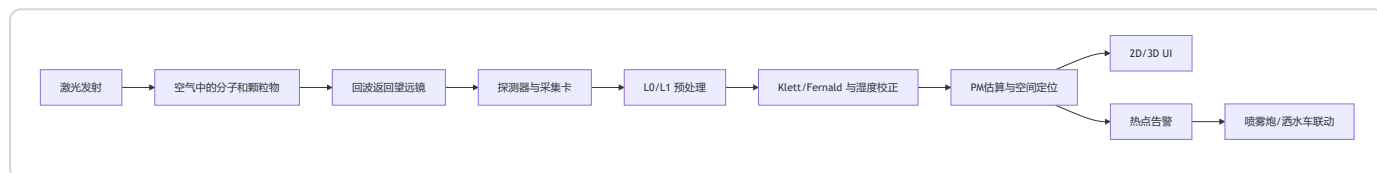
术语	小白解释	你为什么必须懂
激光脉冲	一次很短的激光闪光	它是系统的“探测动作”
回波	被空气和颗粒物打回来的一点光	这是仪器真正测到的信号
后向散射	光被散射后，刚好朝设备方向返回	不返回就收不到
消光	光在传播中被散射和吸收，越来越弱	决定远距离还能不能看见
后向散射系数 β	某一层空气把光打回来的能力	决定这一层“亮不亮”
消光系数 α	某一层空气削弱激光的能力	决定光路损耗有多大
overlap 重叠函数	发射光束和接收视场在近距离有没有完全对上	决定近距离盲区
RCS 距离平方校正	把天然的 R^{-2} 几何衰减先补回来	便于看真实结构
Klett/Fernald	一种经典反演算法	把回波变成消光和后向散射
ENU 坐标	东、北、天的本地三维坐标系	把扫描点投到地图和 3D 场景里

3. 整个系统由哪些部分组成

很多小白会以为“激光雷达就是一个激光器”。其实完全不是。一个能工作的颗粒物激光雷达系统至少包含下面 6 部分：

1. 发射端：激光器、扩束镜、安全快门、能量监测。
2. 接收端：望远镜、滤光片、分光器、探测器。
3. 采集端：模数采样卡、光子计数器、同步时钟。
4. 姿态与定位：GPS、IMU、车体姿态、云台角度编码器。
5. 算法软件：预处理、反演、标定、融合、告警。
6. 显示与控制：2D 图、3D 地图、热点告警、喷雾联动。

用流程图来看会更直观：



你可以把它理解成：

- 激光器负责“问空气一个问题”。
- 望远镜和探测器负责“听空气怎么回答”。
- 算法负责“把回答翻译成人能看懂的数据”。
- UI 和控制系统负责“把结论用图和动作表现出来”。

4. 从一束脉冲到一条距离曲线，中间发生了什么

4.1 为什么时间能换成距离

激光飞得非常快，接近光速 c 。设备发出脉冲后，系统开始计时。某一段空间的回波返回得越晚，说明它离设备越远。

距离公式是：

$$R = \frac{ct}{2}$$

这里的 2 很关键，因为光不是单程，而是：

1. 从设备飞到目标体积。
2. 再从目标体积飞回设备。

所以如果回波晚了 t 秒，真正的单程距离只是一半。

4.2 为什么原始数据不是照片，而是一条曲线

因为设备每次发射只知道“不同时间收到了多强的回波”，所以原始结果更像：

- 横轴是距离。
- 纵轴是回波强度。

如果持续采样很多次，再把时间拼起来，就会得到：

- 时间-高度图。
- 距离-仰角图。
- 方位-距离图。
- 三维体素图。

4.3 为什么近处和远处信号差很多

原因主要有 4 个：

1. 距离越远，几何扩散越明显，信号天然按 R^{-2} 变弱。
2. 激光沿途被空气和颗粒物衰减。
3. 近距离 overlap 不完整，存在盲区。
4. 白天太阳背景光和电子噪声会抬高底噪。

5. 最核心的 LiDAR 方程，必须真正看懂

颗粒物弹性后向散射 LiDAR 的经典形式可以写成：

$$P(R) = CE \frac{O(R)}{R^2} \beta(R) \exp \left[-2 \int_0^R \alpha(r) dr \right]$$

第一次看到这个式子不用怕。你先不要想着推导它，而是先读懂它在说什么。

5.1 一句人话版

这条方程说的是：

设备在距离 R 这一层空气收到的回波强度，等于“设备本身有多强”乘以“这一层有多少东西把光打回来”再乘以“光在来回路上还剩多少”。

5.2 每一项分别是什么意思

符号	含义	你可以怎么理解
$P(R)$	距离 R 处的回波信号	设备真正测到的东西
C	系统常数	仪器整体效率打包值
E	单脉冲能量	这一枪激光打得有多强
$O(R)$	重叠函数	发射和接收在该距离有没有对准
R^{-2}	几何扩散项	距离越远，回波天然越弱
$\beta(R)$	后向散射系数	这一层空气把光打回来的能力
$\alpha(r)$	消光系数	光在传播中被削弱的程度
$\exp[-2 \int_0^R \alpha(r) dr]$	双程传输项	去一趟再回来，最后还剩多少光

5.3 为什么会有一个指数函数 $\exp$

式子里的 $\exp(x)$ 就是 e^x 。如果你对指数函数不熟，可以只把它理解成“衰减开关”。

几个最直观的值：

- $\exp(0) = 1$ ，表示完全不衰减。
- $\exp(-1) \approx 0.368$ ，表示只剩 36.8%。
- $\exp(-2) \approx 0.135$ ，表示只剩 13.5%。

所以：

$$\exp \left[-2 \int_0^R \alpha(r) dr \right]$$

实际上就是在回答一个问题：

光从仪器出发，走到距离 R 的目标层，再从那一层返回仪器，最后还能剩下多少比例？

5.4 这里的积分到底在干什么

积分

$$\int_0^R \alpha(r) dr$$

不是在炫数学，它只是表示：

把 0 到 R 这一路上每一小段空气造成的衰减，全部加起来。

这里的 r 只是一个中间位置变量，你可以理解成“我现在走到哪一段空气了”。

例如你要看 1000 m 处的回波，那就要把 0 m 到 1000 m 之间每一小段空气的衰减全部累加。

5.5 为什么前面要乘以 -2

因为光会穿过同一段大气两次：

1. 去程一次。
2. 回程一次。

所以衰减要算双程，前面自然就是 -2 。

5.6 一个非常粗略的数值例子

假设整条路径上消光系数近似不变：

$$\alpha = 0.1 \text{ km}^{-1}$$

如果目标层在：

$$R = 3 \text{ km}$$

那就有：

$$\int_0^R \alpha(r) dr \approx \alpha R = 0.1 \times 3 = 0.3$$

双程衰减项就是：

$$\exp(-2 \times 0.3) = \exp(-0.6) \approx 0.55$$

这说明什么？

说明激光去一趟再回来之后，大约只剩 55% 的强度还能贡献给回波。

如果空气更脏、雾更大、距离更远，这个数字还会继续掉。

5.7 为什么看起来“远处很弱”不一定代表远处没有污染

这是初学者非常容易犯的判断错误。

远处回波弱，可能有两种完全不同的原因：

1. 那里真的颗粒物少，后向散射小。
2. 那里其实颗粒物很多，但前面路径上的衰减太强，光到那里时已经没剩多少了。

所以你不能直接把“亮度”当成“浓度”。这也是为什么必须做反演，而不是直接目测颜色深浅。

6. 为什么单靠一条回波曲线，不能直接把所有物理量都解出来

方程里最关键的两个未知量是：

- $\beta(R)$ ：后向散射系数。
- $\alpha(R)$ ：消光系数。

问题是：

- 你测到的是 $P(R)$ 。
- 但式子里同时有 α 和 β 。
- 只靠一条单波长弹性回波，方程未知数太多。

所以一定要引入额外假设或额外信息。

最常见的做法就是假设 lidar ratio：

$$S = \frac{\alpha_{\text{aerosol}}}{\beta_{\text{aerosol}}}$$

你可以把它理解成：

先假设某一类气溶胶的“消光”和“后向散射”之间有一个经验比例关系，然后用这个关系把未知数数量降下来。

这也是 Klett / Fernald 反演的基本前提。

7. 什么是 Rayleigh、Mie、偏振，为什么颗粒物主要看 Mie

7.1 Rayleigh 散射是什么

如果散射体远远小于波长，比如空气分子，那么通常更接近 Rayleigh 散射。

它的特点是：

1. 更偏分子层面。
2. 波长依赖很强，短波更明显。
3. 在非常干净的大气里占比更重要。

7.2 Mie 散射是什么

如果颗粒尺寸和波长差不多，比如灰尘、烟粒、雾滴，那通常更接近 Mie 散射。

它的特点是：

1. 和颗粒物关系更直接。
2. 对工地扬尘、道路灰尘、工业粉尘更关键。
3. 也是颗粒物监测里最常见的主角。

7.3 为什么工地和道路场景优先做弹性 Mie 后向散射

因为它有 4 个现实优势：

1. 硬件成熟。
2. 成本相对可控。
3. 算法链条成熟。
4. 对粉尘团、扬尘层、污染羽流特别敏感。

如果你的目标是先把“哪里脏、哪边起尘、喷雾该打向哪里”这件事做出来，那先做颗粒物弹性 LiDAR 是最实际的路线。

8. 为什么第一版不建议直接做气体 DIAL

这不是说 DIAL 不强，而是说它不适合作为小白第一步。

原因很现实：

1. 两个波长都要稳定，而且必须稳定在目标吸收线附近。
2. 激光线宽、波长漂移、温压修正都很敏感。
3. 硬件、校准、法规和安全难度都更高。
4. 软件链路更复杂，调试周期更长。

如果你的实际业务是工地扬尘监管、道路保洁联动、料场抑尘，那最优先的路线不是“上来就做最难”，而是：

1. 先把颗粒物和粉尘空间监测跑通。
 2. 先把闭环控制跑通。
 3. 等团队掌握了扫描、反演、标定和 UI，再考虑是否追加单一气体功能。
-

9. 工程上应该怎么部署：固定式和车载式为什么都重要

9.1 固定式高点扫描

固定式一般装在：

- 楼顶。
- 塔吊平台。
- 高杆。
- 园区边缘高点。

它的优势是：

1. 视野高，覆盖大。
2. 适合做持续监控。
3. 适合看工地整体扬尘扩散趋势。
4. 适合联动固定喷雾炮。

它比较像一个“区域总控哨兵”。

9.2 车载式移动扫描

车载式一般装在：

- 洒水车。
- 雾炮车。
- 环卫巡检车。
- 园区巡逻车。

它的优势是：

1. 可以贴近路面和污染源。
2. 可以沿着道路或厂区边界做移动扫描。
3. 容易发现局部热点和死角。
4. 可以边走边扫边处置。

它更像一个“流动侦察兵”。

9.3 为什么两者组合最好

因为它们解决的是不同尺度的问题：

- 固定式看全局。
- 车载式看局部。
- 固定式适合盯守。
- 车载式适合追热点。

对真正要落地的工地或园区系统来说，最理想的架构往往不是二选一，而是组合：

1. 高点固定式负责区域大范围扫描和告警。
 2. 车载式收到任务后去热点附近复扫。
 3. 结果再回传平台，形成闭环证据链。
-

10. 硬件怎么选，小白先抓住哪些原则

10.1 波长怎么选

如果只做颗粒物和扬尘，常见路线大致如下：

路线	常见波长	优点	缺点	适用阶段
微脉冲/云高仪路线	905 / 910 / 1064 / 1550 nm	相对更安全、连续运行容易	定量能力较弱，近距离重叠问题突出	入门和工程首版
532 nm 弹性 LiDAR	532 nm	对颗粒物敏感，资料多	安全压力大，法规要求高	进阶工程版
多波长偏振	355 / 532 / 1064 nm	分类能力更强	光路复杂、成本高	高级版

10.2 接收端为什么也很重要

很多人会把注意力全放在激光器上，其实接收端同样关键。

接收端决定了：

- 1. 你能收回多少微弱光子。
- 2. 白天背景光能压掉多少。
- 3. 多通道能不能分开。
- 4. 远距离和低浓度目标能不能看见。

10.3 近距离盲区为什么总被反复强调

因为发射光束和接收视场在很近的地方不一定完全重合。

这意味着：

- 1. 很近的粉尘可能明明就在设备前面。
- 2. 但系统却收不到完整回波。
- 3. 结果就是近距离数据偏低甚至失真。

这部分必须靠 overlap 校正或光机设计优化解决。

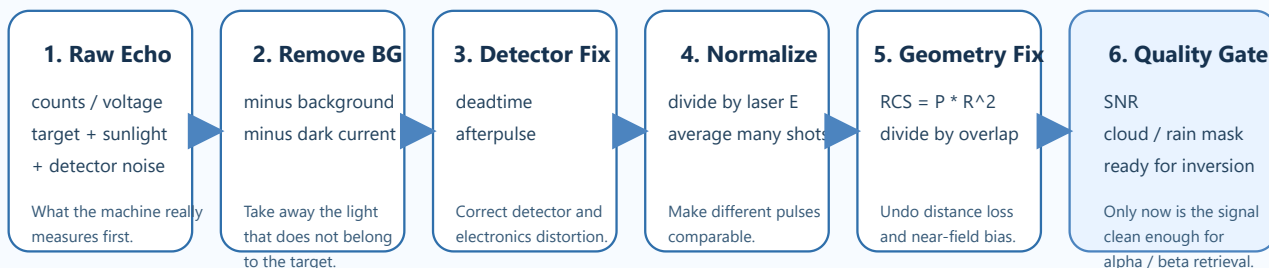
11. 颗粒物版本的数据处理到底怎么走

这一节是整份文档里最重要的工程部分之一。因为真正把系统做出来时，你面对的不是“一个公式”，而是一整条数据流水线。

11.0 如果你数学基础比较弱，先看这张总流程图

Algorithm Picture 1: From Raw Echo To Clean Signal

Read it left to right. Each box removes one kind of non-target error before inversion starts.



Key Message

Preprocessing is not cosmetic. It removes non-target effects first, so later inversion does not turn noise into fake physics.

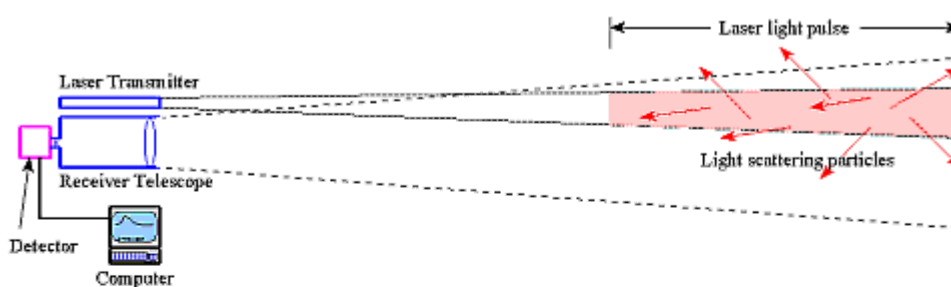
你可以先不要急着记公式，先把这张图读顺：

1. 最左边是机器真正测到的原始回波，它里面既有目标信息，也有各种噪声。
2. 中间每一个方框都在解决一种具体误差，例如背景光、探测器失真、发射能量波动、近距离重叠不足。
3. 最右边的意思是：只有把这些非目标因素先拿掉，后面才值得做反演。

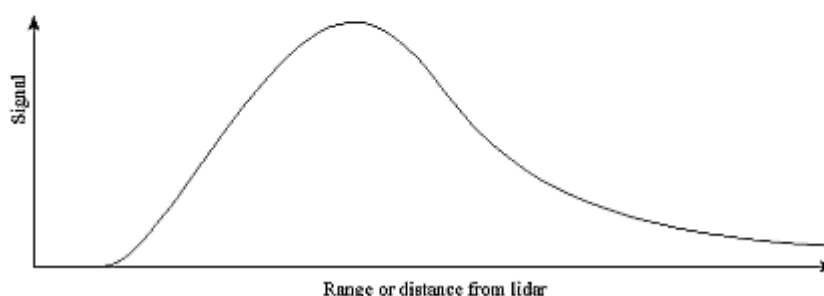
如果你现在只想抓住一句话，那就是：

算法链前半段不是在“炫技术”，而是在努力回答一个更朴素的问题：我看到的回波，到底有多少是真的空气和颗粒物，多少只是设备和环境带来的假信号。

如果你更想先看一张网上现成、最经典的基础示意图，可以看 NOAA 这张：



(a) Basic components of lidar



(b) Typical lidar signal as a function of range

这张图特别适合数学基础弱的读者，因为它同时把两件事画出来了：

- 1. 左边是空间过程：激光脉冲打出去，沿路不断有微弱回波返回。
- 2. 右边是数据结果：返回信号会随着距离变化形成一条曲线。
- 3. 这张图把“空间里发生了什么”和“电脑里最后看到什么”直接连在一起了。

你读这张图时，建议只抓住 3 个问题：

- 1. 为什么越远回波越晚回来。
- 2. 为什么回波曲线不是平的，而会有起伏和峰值。
- 3. 为什么曲线里的高值区，往往对应空气里有更多散射体的区域。

来源：NOAA Chemical Sciences Laboratory 的 LIDAR 原理页，适合做入门直觉图。

老师讲课版：这张图请按 4 眼来看

第 1 眼，只看左边，不看右边。

你先把自己想象成站在设备旁边，看到一束很短的激光脉冲打出去。它不是一下子照亮整片天空，而是像一个很短的光包沿着前方飞出去。

第 2 眼，继续只看左边，注意“沿路每一段空气都会给一点点回波”。

空气分子、颗粒物、粉尘、烟雾，都可以把极少量光散射回来。离设备近的那一段空气，会先把回波送回来；离设备远的那一段空气，会更晚才把回波送回来。

第 3 眼，再去看右边曲线。

右边这条曲线，本质上就是把“不同距离上返回了多少光”按距离排成一列。也就是说：

- 1. 左边空间里每一小段空气。
- 2. 在右边曲线上都会对应一个自己的位置。
- 3. 哪一段空气散射更强，曲线在对应距离上就更高。

第 4 眼，把左右两边连起来。

这一步最关键。你要建立一个稳定直觉：

左边看到的是物理世界里光怎么走，右边看到的是电脑把这些返回光按距离排好之后的结果。

如果你只能记住一句最核心的话，那就是：

空间里哪一段空气更容易把光打回来，曲线在那个距离位置就更容易鼓起来。

这就是后面所有预处理、反演、识别算法的起点。因为算法并不是凭空制造信息，它只是想办法把这条曲线里的结构，重新翻译回空间里的粉尘分布。

建议把数据链分成 5 个层级，这样工程上最好管理。

级别	内容	典型数据形态	小白理解
L0	原始 ADC / photon counts / 元数据	一维波形、二维时距矩阵、角度序列、姿态序列	机器刚吐出来的原始记录
L1	背景扣除、暗电流、死时间、能量归一化、RCS	已清洗的 profile	先把脏信号洗干净
L1.5	attenuated backscatter、depol、云雨 mask、SNR	可展示的物理图层	可以先给人看图了

级别	内容	典型数据形态	小白理解
L2	backscatter、extinction、PM 估算、层顶层底	物理量剖面 and 扫描面产品	真正进入业务指标
L3	2D/3D 网格、热点轨迹、统计报表、告警事件	体素、栅格、事件表、质心点	平台和联动控制直接消费

11.1 一次原始采样到底长什么样

很多初学者会以为设备每次输出的是一张图。其实不是。设备最底层拿到的通常是一串“按距离分箱的回波数组”。

最基础的一次 profile 往往包含：

- 1. 时间戳。
- 2. 通道编号。
- 3. 发射能量。
- 4. 方位角和仰角。
- 5. 回波数组。
- 6. 设备状态。
- 7. GPS 和 IMU 信息。

一个很简化的记录可以长这样：

```
{
  "timestamp": "2026-05-27T10:23:15.120Z",
  "channel": "532_elastic",
  "azimuth_deg": 35.0,
  "elevation_deg": 12.0,
  "laser_energy_mj": 1.82,
  "range_resolution_m": 7.5,
  "signal_counts": [128, 124, 119, 116, 109, 98, 91],
  "gps": [121.4737, 31.2304, 28.5],
  "imu": [0.2, -0.5, 37.1]
}
```

这里的 signal_counts 只是示意。真实系统里它一般会 longer，常常是几百到几千个 range bin。

如果做的是连续垂直观测，数据更像：

$$\text{signal}[t, r]$$

也就是“时间 × 距离”的二维矩阵。

如果做的是 PPI 或 RHI 扫描，数据更像：

$$\text{signal}[\text{scan}, \text{angle}, r]$$

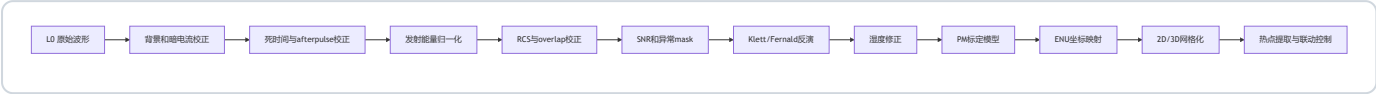
如果做的是车载扫描，还要再叠加：

$$\text{signal}[t, r] + \text{pose}[t] + \text{gps}[t]$$

也就是说，同一时刻的回波必须和同一时刻的姿态、位置严格绑定，否则后面做三维地图时就会飘。

11.2 从 L0 到 L3 的全链路应该怎么理解

最推荐你记住的是下面这条链：



这个流程里，每一步都不是“可有可无的优化”，而是在解决一种明确的误差来源。

11.3 为什么预处理不能省

因为原始回波里混着很多不属于目标的信息，例如：

1. 太阳背景光。
2. 探测器暗电流。
3. photon counting 死时间效应。
4. 发射能量波动。
5. afterpulse 假回波。
6. 近距离 overlap 缺陷。
7. 云、雨、雾、强反射导致的异常点。

如果不先清理这些问题，后面的反演几乎一定会偏，而且偏得并不直观。

11.4 小白先记住的预处理顺序

1. 时间同步和角度同步。
2. 背景扣除。
3. 暗电流扣除。
4. 死时间和 afterpulse 校正。
5. 发射能量归一化。
6. 时间平均和距离重采样。
7. 距离平方校正。
8. overlap 校正。
9. SNR 估计。
10. 云、雨、雾、异常点 mask。

11.5 每一步到底在做什么

第 1 步：时间同步和角度同步

这是整条数据链最容易被忽视的地方。

你必须保证：

1. 这一条回波对应的是哪一次激光发射。
2. 这一条回波对应的是哪个方位角和仰角。
3. 如果是车载，还要知道这一刻车在哪里、姿态如何。

如果时间对不上，后果会非常严重：

1. 地图投影错位。
2. RHI 和 PPI 图像撕裂。
3. 热点看起来像在跳动。
4. 喷雾指令会打偏。

第 2 步：背景光扣除

背景光主要来自太阳散射、城市光、电子底噪。最简单的做法是取远距离无有效回波的尾部区间，求一个平均背景值：

$$B = \frac{1}{N} \sum_{i=r_1}^{r_2} P_{\text{raw}}(i)$$

然后：

$$P_1(R) = P_{\text{raw}}(R) - B$$

这一步的直观意义是：

先把“不属于目标的常亮底噪”减掉。

第 3 步：暗电流扣除

探测器即使没有光，也可能自己产生电信号，这部分叫暗电流。它通常通过实验室暗场测量或定期关快门采集得到：

$$P_2(R) = P_1(R) - D(R)$$

其中 $D(R)$ 可以是一个常数，也可以是随距离变化的标定曲线。

第 4 步：死时间校正

如果用 photon counting，探测器或计数电子学在记录了一个光子之后，需要很短一段恢复时间，这段时间叫死时间 τ 。高计数率时会出现“漏记数”。

非瘫痪模型下，一个常见近似是：

$$N_{\text{true}} = \frac{N_{\text{obs}}}{1 - \tau N_{\text{obs}}}$$

意思是：

- 观测计数 N_{obs} 偏低。
- 真值 N_{true} 要往上修正。

如果不做这一步，近距离强信号区会被压扁。

第 5 步：afterpulse 校正

afterpulse 可以理解为探测器或电子链路在主脉冲之后留下的“拖尾假信号”。

它常用一条实验标定得到的参考曲线来扣除：

$$P_3(R) = P_2(R) - A(R)$$

其中 $A(R)$ 是 afterpulse 模板。

第 6 步：发射能量归一化

每一枪激光的能量不可能完全一样。为了让不同 profile 可以直接比较，通常要做能量归一化：

$$P_4(R) = \frac{P_3(R)}{E_{\text{laser}}}$$

如果没有这一步，你看到的波动可能不是空气变了，而只是激光器输出抖了。

第 7 步：时间平均和距离重采样

单发回波通常很噪，所以工程里经常把若干发脉冲平均成一个 profile。

例如 1000 Hz 重复频率、每 1 秒输出一帧，那就是平均 1000 发。平均后信噪比会提升，直观上近似满足：

$$\text{SNR} \propto \sqrt{N_{\text{shots}}}$$

意思是：

- 平均 4 倍发数，SNR 大约提高 2 倍。
- 但时间分辨率会下降。

所以工程上永远是在“稳定”和“灵敏”之间找平衡。

第 8 步：距离平方校正 RCS

最基础的一步就是：

$$\text{RCS}(R) = P_4(R)R^2$$

它的作用不是“直接得到浓度”，而是先把最显眼的几何扩散趋势补回来，让曲线更容易观察结构变化。

你可以理解成：

先把“远处天然更暗”这件事粗略补偿掉，再去看哪里真的是结构变化。

第 9 步：overlap 校正

近距离时，发射光束和接收视场没有完全重合，所以实际能收回的回波比例偏低。通常用一个重叠函数 $O(R)$ 来描述：

$$P_{\text{corr}}(R) = \frac{P_4(R)}{O(R)}$$

如果 $O(R)$ 在近距离小于 1，而你又不校正，那么近端粉尘会被严重低估。

第 10 步：SNR 估计和质量标志

一个很常见的简化写法是：

$$\text{SNR}(R) = \frac{P_{\text{signal}}(R)}{\sigma_{\text{noise}}(R)}$$

或者在 photon counting 近似下：

$$\text{SNR}(R) \approx \frac{N(R)}{\sqrt{N(R) + N_{\text{bg}}}}$$

SNR 的作用非常大，因为后面很多反演步骤只应该在 SNR 足够高的区间做。

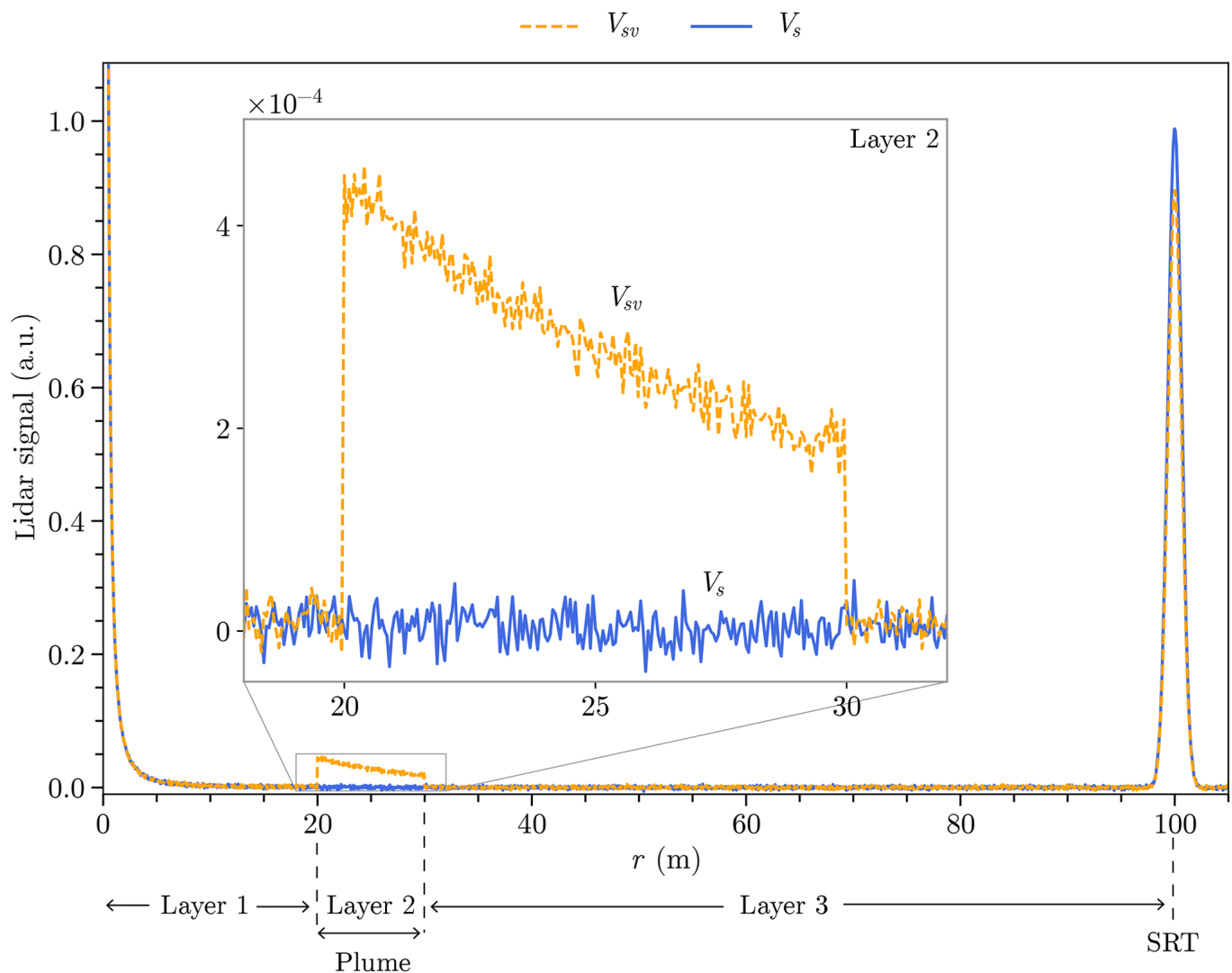
11.6 一个工程上能落地的预处理伪代码

```
def preprocess_profile(raw_signal, ranges, laser_energy, background_slice, overlap_curve):
    background = raw_signal[background_slice].mean()
    signal = raw_signal - background
    signal = deadtime_correct(signal)
    signal = signal - afterpulse_template(ranges)
    signal = signal / max(laser_energy, 1e-9)
    signal = range_resample(signal, ranges)
    rcs = signal * ranges**2
    corrected = rcs / np.maximum(overlap_curve, 1e-6)
    snr = estimate_snr(corrected)
    mask = build_quality_mask(corrected, snr)
    return corrected, snr, mask
```

这段伪代码传达的核心思想只有一句话：

预处理的本质不是“美化数据”，而是把不属于目标本身的误差因素尽量先拿掉。

如果你想看一张“有羽流”和“没羽流”时信号会差成什么样的图，下面这张理论示意特别有帮助：



这张图怎么读：

1. 蓝线可以理解成“没有额外羽流时”的参考信号。
2. 橙色虚线可以理解成“路径上多了一层气溶胶羽流后”的信号。
3. 中间放大的小窗最重要，它告诉你真正的差异往往发生在某个局部距离段，而不是整条曲线同时一起变化。
4. 最右边很高的尖峰是表面参考目标回波，小白先不用深究它的光学定义，只要知道它是一个非常强、非常明显的参考峰。

这张图对理解算法特别有用，因为它让你直观看见：

算法不是从空气里“凭空算浓度”，而是先从“有无羽流时曲线到底差在哪里”开始读信息。

老师讲课版：这张图不要一上来全看，按顺序拆开看

第 1 步，先只看横轴。

你可以把横轴理解成“离设备有多远”。现在先不要管单位细节，只要知道曲线上的每个位置，都对应前方某个距离段。

第 2 步，再只看蓝线和橙线是不是完全重合。

如果两条线几乎重合，说明有无羽流差别不大；如果某个距离段开始明显分开，就说明那个距离段很可能出现了额外的散射或衰减结构。

第 3 步，再去看看中间的小放大窗。

小白最容易忽略的恰恰是这里。作者单独开一个放大窗，就是想告诉你：

1. 真正有业务意义的差异，有时并不大。
2. 但只要它持续出现在同一个距离段，就不能当成随机噪声忽略。
3. 算法后面做差分、做阈值、做识别，本质上就是在盯这种局部但持续的差别。

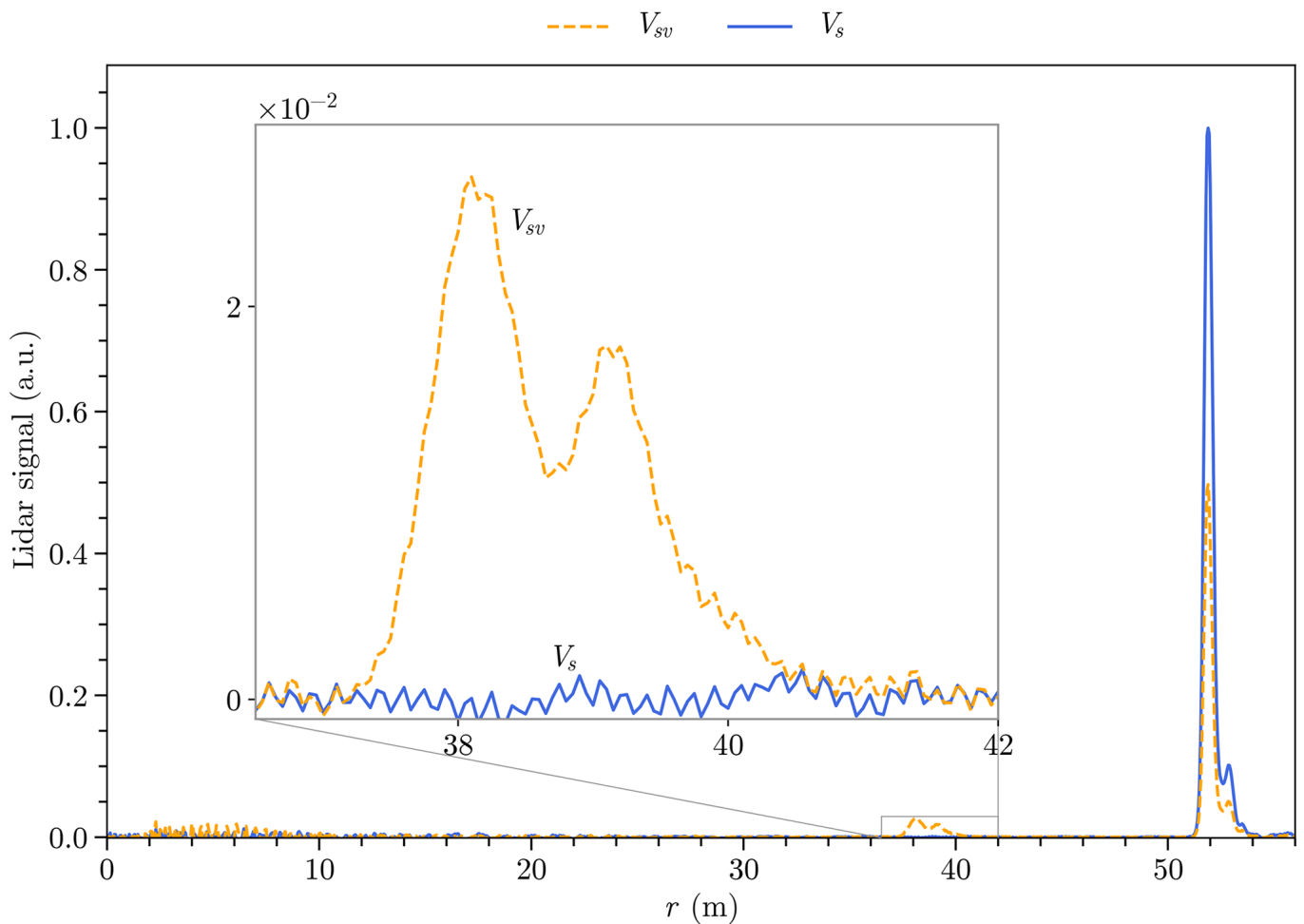
第 4 步，最后再看最右边的大尖峰。

这个尖峰可以暂时理解成一个很强的参考回波。它提醒你一件重要事实：真实曲线里并不只有羽流信息，还会混着参考目标、边界回波、仪器和场景条件带来的强响应。

所以这张图真正教你的，不只是“橙线比蓝线高一点”，而是下面这句话：

算法要做的，是先在整条复杂曲线里找出“哪一段差异是稳定且有物理意义的”，再把这段差异解释成空间中的羽流或颗粒物变化。

如果你再看一张真实测量图，会发现真实世界的曲线比理论图更乱、更粗糙：



这张真实图最值得你注意的是：

1. 真实曲线不像理想示意图那么平滑。
2. 噪声、小起伏、局部尖峰都会出现。
3. 但只要羽流足够明显，橙线和蓝线在关键距离段仍然会拉开差距。

这正是为什么前面那些背景扣除、平均、SNR、mask 不能省，因为：

真实数据永远比教材图更脏，预处理就是把这些脏东西尽量压下去，让真正的羽流差异留下来。

老师讲课版：这张真实图最适合拿来理解“为什么算法必须分步骤做”

你可以按下面这个顺序读：

1. 先不要追求看懂每一个抖动，只先看蓝线和橙线的大体走势。
2. 再问自己：它们在哪一段距离开始出现系统性分离，而不是偶然碰一下就分开。
3. 如果只是单个尖点，很可能是噪声、瞬时扰动或局部异常；如果是一整段范围都分开，就更值得怀疑那里有真实羽流。

对初学者来说，这一步非常关键，因为很多人第一次看真实数据时会犯两个极端错误：

1. 要么把每一个小尖峰都当成真实污染事件。
2. 要么因为曲线太乱，干脆觉得什么也看不出来。

正确的做法是：

不看单个点，先看一整段距离里有没有连续、稳定、成片的差异。

这也正是时间平均、空间平均、SNR 门限和质量标志存在的原因。它们不是为了把图修得好看，而是为了帮助你区分“稳定结构”和“随机毛刺”。

如果把它翻译成工程动作，就是：

1. 先压噪声。
2. 再找连续异常距离段。
3. 然后才去做反演和 PM 估算。

来源：Gaudfrin 等人在 Atmospheric Measurement Techniques 2020 的公开论文图，CC BY 4.0，可用于教学说明。

12. 算法部分展开讲：从回波到消光、PM 和热点

这一节开始，我们不再停留在“知道有哪些算法”，而是逐步讲清楚数据是怎么一步一步被变成业务指标的。

12.0 先把 LiDAR 方程看成 4 个功能块

Algorithm Picture 2: Read The Lidar Equation As Four Functional Blocks

$$P(R) = [C \ E] * [O(R) / R^2] * [\text{beta}(R)] * [\exp(-2 * \int \alpha \text{ dr})]$$

Block 1: Instrument

$$C * E$$

laser pulse energy
optics efficiency
detector gain

If this changes, the whole curve
may move up or down.

Block 2: Geometry

$$O(R) / R^2$$

near-field overlap
distance spreading

This is why far range is dim and
near range can be biased.

Block 3: Scatter Back

$$\text{beta}(R)$$

how much a layer sends
light back to the lidar

This block is strongly linked to
aerosol or dust presence.

Block 4: Transmission

$$\exp(-2 * \int \alpha \text{ dr})$$

outgoing loss
incoming loss

A strong distant plume can still
look weak after round-trip loss.

Why inversion is hard: $\text{beta}(R)$ and $\alpha(R)$ are both unknown, but the lidar only measures $P(R)$.

如果你一看到公式就紧张，可以先完全不碰推导，只看这张图。

这张图真正想表达的是：

1. 回波强弱不只取决于颗粒物本身。
2. 仪器强不强、几何扩散大不大、这一层会不会把光打回来、来回路上损耗有多大，都会同时影响结果。
3. 反演之所以难，是因为你只测到了一个回波 $P(R)$ ，但里面至少混着后向散射和消光这两个未知量。

所以对数学基础弱的读者来说，最重要的不是先背公式，而是先记住这个因果关系：

曲线变高，不一定只是颗粒物变多，也可能是仪器、几何或传播条件在变化。

12.1 第一个能看的物理量：attenuated backscatter

很多系统不会一上来就展示绝对定标后的 backscatter，而是先展示 attenuated backscatter，也就是“受传播衰减影响的后向散射图层”。

它常常和 RCS 接近，但会带一个整体定标因子：

$$\beta'_{\text{att}}(R) \propto P(R)R^2$$

为什么这个量很常见：

1. 它比原始信号更容易看结构。
2. 它不需要立刻把所有绝对标定都做完。
3. 时间-高度图和扫描剖面一般直接用它来做主视觉层。

12.2 参考区段为什么重要

Klett/Fernald 不是凭空把所有东西都解出来，它必须有一个边界条件。最常见的做法是在远端选一个相对稳定、相对干净的参考区段 R_c 。

理想的参考区段通常满足：

1. SNR 还够用。
2. 没有明显浓烟或强尘团。
3. 不在云、雨、雾强回波里。
4. 与分子背景或历史统计更接近。

简单说：

你得先找一段“相对可信”的空气，算法才能从这里开始往回推。

12.3 Klett/Fernald 到底在算什么

单波长弹性 LiDAR 的难点在于：

1. 后向散射系数 $\beta(R)$ 未知。
2. 消光系数 $\alpha(R)$ 也未知。
3. 只看一条回波曲线，未知量太多。

所以需要引入 lidar ratio：

$$S = \frac{\alpha_{\text{aerosol}}}{\beta_{\text{aerosol}}}$$

在很多工程实现里，会假设 S 在一段距离内近似恒定，或者分区段取不同常数。

12.4 Klett 反演的连续形式怎么理解

Klett 思路的关键是：

1. 把回波先转换成更适合处理的形式。
2. 给出远端参考边界值。
3. 从参考端往近端递推。

常见写法之一可概念化表示为：

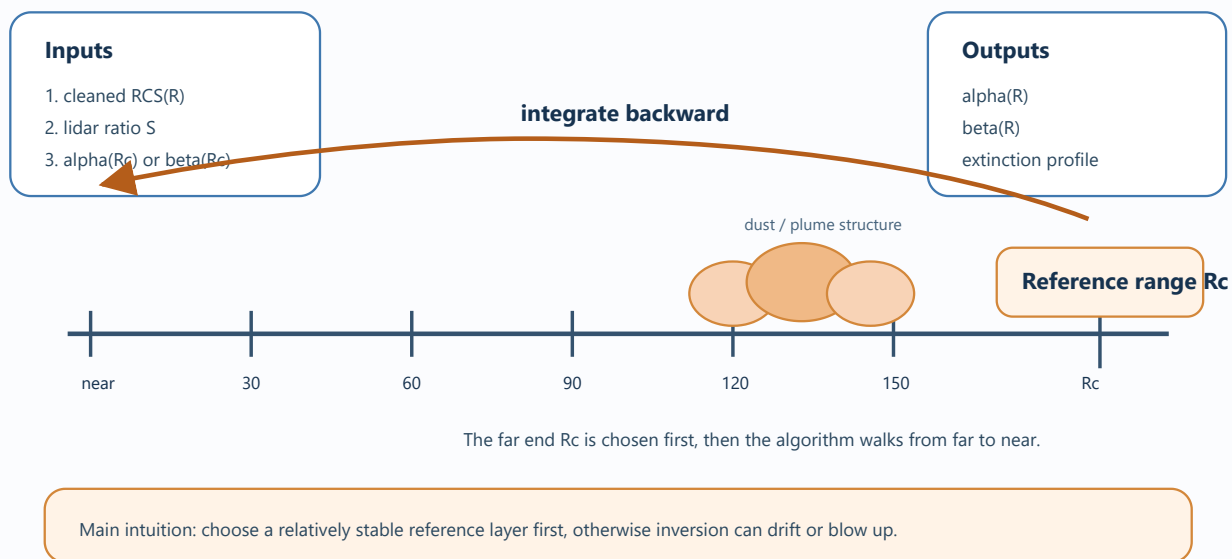
$$\alpha(r) = \frac{\exp\left[\frac{S(r)-S(R_c)}{k}\right]}{\frac{1}{\alpha(R_c)} + \frac{2}{k} \int_r^{R_c} \exp\left[\frac{S(r')-S(R_c)}{k}\right] dr'}$$

初学者不一定要马上推导这条式子，但一定要理解它在工程上的逻辑：

1. 先给一个参考端边界值。
2. 把信号和假设的 lidar ratio 结合起来。
3. 再从远端一步一步往回恢复每个距离上的光学量。

如果你对“为什么一定要找参考区段”还是没有直觉，可以先看下面这张图：

Algorithm Picture 3: Why Klett Starts From A Reference Range R_c

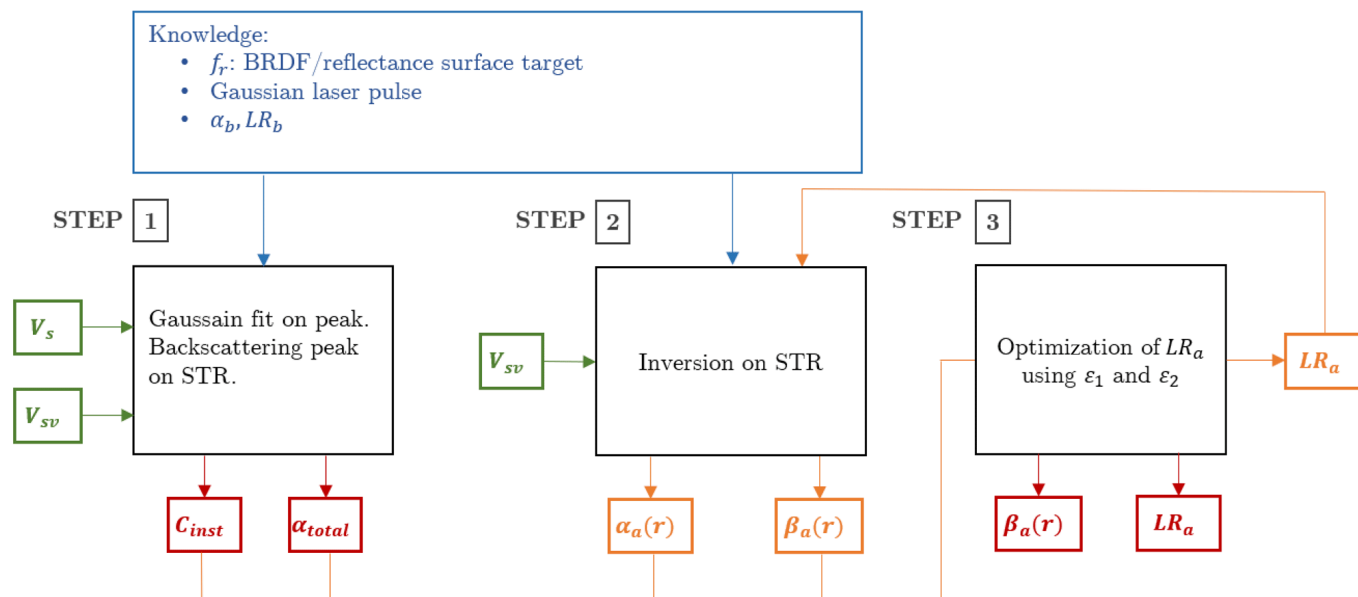


你可以把它理解成：

1. 算法并不是从近处瞎猜到远处。
2. 它通常先在远端找一个相对稳定、相对干净的参考层。
3. 然后从这个参考层往近端一格一格递推。

这也是为什么参考区段选坏了，后面整条反演曲线都会跟着漂。

如果你想看一张“论文作者自己怎么把反演步骤画成流程图”的现成图，可以看这张：



这张图虽然是论文图，但你完全可以把它翻译成成人话来读：

1. Step 1：先盯住参考目标或参考峰，把仪器常数和总衰减这类基础量先估出来。
2. Step 2：带着这些基础量，先做一轮初始反演，拿到第一版的后向散射和消光信息。
3. Step 3：继续做优化，让模型生成的信号和真实测到的信号尽量对上。

对数学基础弱的读者来说，这张图最大的价值是让你明白：

反演不是一次按下按钮就出结果，而是“先给起点，再算第一版，再不断逼近更合理结果”的过程。

也就是说，Klett / Fernald 这类方法更像“带约束的反推”，而不是普通的直接代公式。

老师讲课版：把这张流程图当成“做一道反推题”的步骤表

你可以把它想成下面这个过程：

1. 先拿到一条真实测得的回波曲线。
2. 但这条曲线里面混着仪器影响、传播损耗和颗粒物信息，不能直接读成浓度。
3. 所以算法必须先给自己找一个起点，也就是参考峰、参考区段或者某种边界条件。
4. 有了这个起点，先做第一版估计。
5. 第一版通常还不够好，所以还要继续比较“算出来的信号”和“真实测到的信号”像不像。
6. 如果还不像，就继续调，直到逼近一个更合理的结果。

如果你把这张图用最朴素的话重讲一遍，其实只有三句话：

1. 先定起点。
2. 再反推中间过程。
3. 最后检查推出来的结果能不能解释原始数据。

这就是为什么很多反演算法看上去公式很复杂，但工程实现时可以理解成一个很直白的闭环：

用假设去解释测量，如果解释得不够好，就继续修正假设，直到解释得更合理。

对零基础读者来说，只要你先接受“反演是反推，不是直接测量”，后面的公式就不会那么吓人了。

来源：Gaudfrin 等人在 Atmospheric Measurement Techniques 2020 的公开论文 Figure 2, CC BY 4.0。

12.5 离散化后程序到底怎么做

真实软件不会写连续积分，而是按距离分箱递推。

假设距离序列是：

$$R_0, R_1, \dots, R_n$$

每一小段的间隔是：

$$\Delta R_i = R_{i+1} - R_i$$

那么程序常做的事就是：

1. 在最远端给定 α_n 或 β_n 。
2. 利用第 $i + 1$ 个 bin 的结果，计算第 i 个 bin。
3. 一直往前推到近端。

一个帮助理解的简化版本如下：

```

import numpy as np

def klett_inversion(rcs, ranges, lidar_ratio, alpha_ref):
    rcs = np.asarray(rcs, dtype=float)
    ranges = np.asarray(ranges, dtype=float)

    alpha = np.zeros_like(rcs)
    beta = np.zeros_like(rcs)

    alpha[-1] = alpha_ref
    beta[-1] = alpha_ref / lidar_ratio

    for i in range(len(ranges) - 2, -1, -1):
        dr = ranges[i + 1] - ranges[i]
        trans = np.exp(-2.0 * alpha[i + 1] * dr)
        beta[i] = max(rcs[i] / max(trans, 1e-12), 1e-12)
        alpha[i] = lidar_ratio * beta[i]

    return alpha, beta

```

这段代码不是严格科研版，但它对初学者有两个好处：

1. 能看清楚“从远往近递推”的结构。
2. 能直观看到 α 和 β 是如何互相耦合的。

12.6 lidar ratio 对结果影响有多大

这是工程里非常关键的敏感参数。

如果 S 取小了：

1. 算出来的消光偏小。
2. PM 估算可能整体偏低。

如果 S 取大了：

1. 算出来的消光偏大。
2. 远端容易放大噪声。
3. 热点阈值判断可能偏激进。

所以常见做法是：

1. 先给一个行业经验范围。
2. 再根据地面 PM 站和历史样本反调。
3. 必要时按天气类型或源类型分场景取值。

12.7 为什么反演前后都要平滑

因为原始噪声在指数运算和递推里很容易被放大。

常见方法有：

1. 距离向滑动平均。
2. 时间向滑动平均。
3. Savitzky-Golay 滤波。
4. 小波或正则化平滑。

原则不是“磨到好看”，而是：

1. 不破坏真实羽流边界。
2. 不放大噪声尖峰。
3. 让结果足够稳定可用于阈值判断。

12.8 湿度修正为什么必须进入主流程

颗粒物会吸湿。空气越潮，散射越强，所以只看光学量很容易把“湿了”误判成“浓了”。

常见的工程修正思想可以写成：

$$\alpha_{\text{dry}}(R) = \alpha(R) \cdot \left[1 - \frac{RH(R)}{100} \right]^{\gamma}$$

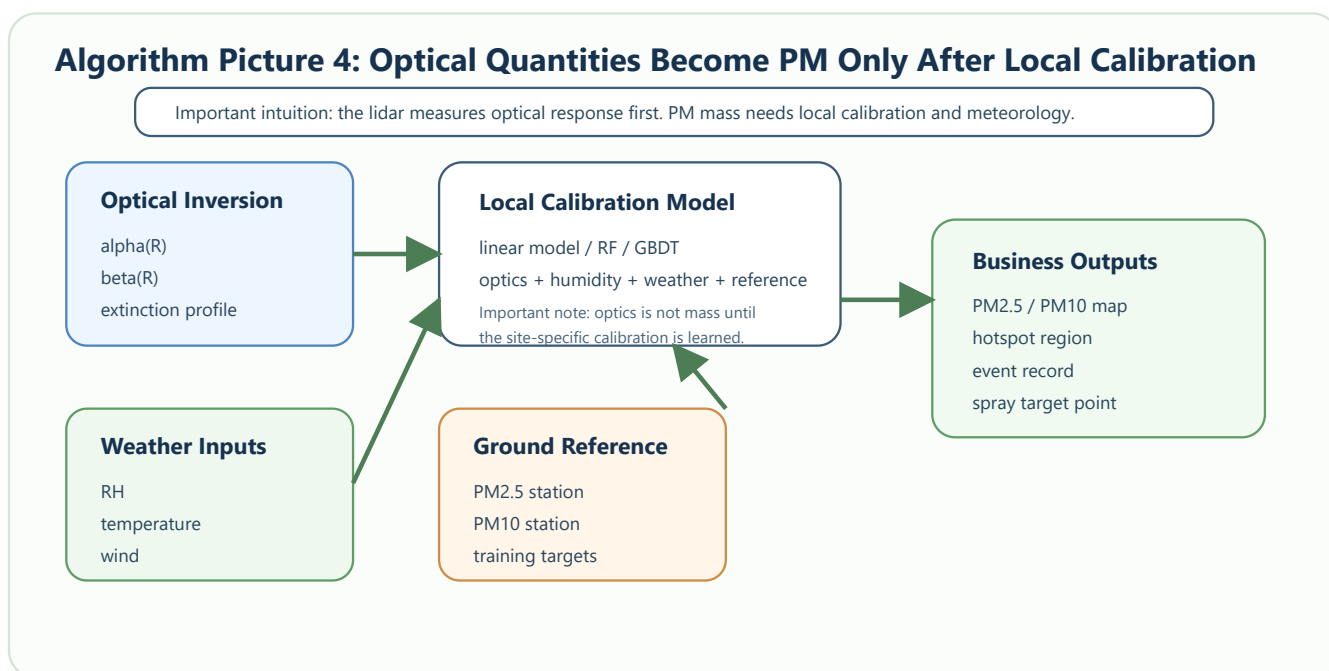
你可以这样理解：

1. $\alpha(R)$ 是未经干化的光学消光。
2. RH 越高，修正幅度越明显。
3. γ 由本地经验决定，不同地区可能不同。

这一步的目的不是把湿度彻底“消灭”，而是尽量让光学量更接近颗粒物本身的干质量特征。

12.9 PM2.5 / PM10 估算到底怎么做

在真正做 PM 估算之前，最建议你先把下面这张图看懂：



这张图想强调一个对初学者最重要的事实：

1. LiDAR 先测到的是光学响应，不是质量浓度。
2. 光学量要先经过湿度和气象修正。
3. 再结合本地地面 PM 参考站，才能训练出适合这个场地的标定模型。
4. 最后业务系统真正拿去告警和联动的是 PM 地图、热点事件和控制目标，而不是单纯的消光曲线。

如果只记一句话，就是：

光学量不等于 PM 质量浓度，中间一定还隔着“本地标定”这一层。

到了这一步，很多初学者以为“有了消光系数就等于有了 PM”。实际上还差最后一层标定。

最简单的经验模型可以写成：

$$PM_{2.5} = a \cdot \alpha_{dry} + b \cdot RH + c \cdot T + d$$

更完整一点的模型会把多个特征一起送进去：

$$PM_{2.5} = f(\alpha, \beta, RH, T, WS, WD, hour, season, aerosol_type)$$

其中：

1. WS 是风速。
2. WD 是风向。
3. $hour$ 和 $season$ 用来吸收日周期和季节性差异。

12.10 PM 标定的训练和上线流程

一个实用的工程流程通常是：

1. 取和地面 PM 站时间对齐的数据。
2. 把 LiDAR 的低层若干 range bin 做平均或加权平均。
3. 提取同一时刻的湿度、温度、风速风向。
4. 构造训练样本。
5. 用线性回归、岭回归、随机森林或梯度提升树训练。
6. 留出验证集评估偏差和相关系数。
7. 模型上线后持续监控漂移。

注意一个关键点：

LiDAR 看到的是一段空间，而地面 PM 站只测一个点，所以训练时必须考虑代表性高度和空间匹配问题。

12.11 一个简单但实用的 PM 推理流程

```
def infer_pm(alpha, beta, rh, temp, wind_speed, model):  
    alpha_dry = humidity_correct(alpha, rh)  
    features = [alpha_dry, beta, rh, temp, wind_speed]  
    return model.predict([features])[0]
```

虽然这段代码很短，但它背后其实对应着完整的数据治理逻辑：

1. 先反演光学量。
2. 再修正湿度。
3. 再和气象量融合。

4. 最后才交给 PM 模型。

12.12 怎么从 PM 场里提取热点

当你已经有扫描面的 PM 或消光场之后，热点提取通常分成 4 步：

1. 设阈值，例如 PM2.5 大于某阈值，或消光高于背景均值若干倍标准差。
2. 做形态学清理，去掉孤立噪点。
3. 做连通域分析，把连续高值区域作为一个 plume 或 dust patch。
4. 计算每个连通域的面积、强度、质心、最大值。

如果用加权质心，一个常见写法是：

$$x_c = \frac{\sum_i w_i x_i}{\sum_i w_i}, \quad y_c = \frac{\sum_i w_i y_i}{\sum_i w_i}, \quad z_c = \frac{\sum_i w_i z_i}{\sum_i w_i}$$

这里的 w_i 可以是 PM 值、消光值或后向散射值。

12.13 算法链里每个输出分别给谁用

你可以把算法链想成一条加工流水线：

1. L1 给算法工程师和质量控制用。
2. L1.5 给值班人员看 quicklook 和异常。
3. L2 给业务模型和告警系统用。
4. L3 给地图、3D、报表和喷雾联动用。

这意味着不同页面不应该直接乱读底层原始数据，而应该各自消费适合自己的那一级产品。

13. 从极坐标到地图坐标，再到 UI 图层

前面讲的是“怎么算出光学量和 PM”，这一节讲“怎么算出它到底在空间里的哪个位置”。

13.1 激光雷达天然输出的是极坐标

激光雷达最直接测到的是：

1. 距离 R 。
2. 方位角 azimuth。
3. 仰角 elevation。

也就是说，每一个数据点本质上都是一根射线上的一个小体元，而不是地图上的现成经纬度点。

13.2 固定式系统怎么转成 ENU

最常用的本地坐标系就是 ENU：

1. E : East, 东。
2. N : North, 北。
3. U : Up, 天。

最基础的转换公式是：

$$\begin{aligned}x &= R \cos(\text{elevation}) \sin(\text{azimuth}) \\y &= R \cos(\text{elevation}) \cos(\text{azimuth}) \\z &= R \sin(\text{elevation})\end{aligned}$$

这一步做完后，你就可以把每个 range bin 放到本地三维场景里。

下面是一个很基础的转换示意：

```
import numpy as np

def polar_to_enu(ranges, azimuth, elevation):
    az = np.deg2rad(azimuth)
    el = np.deg2rad(elevation)

    x = ranges * np.cos(el) * np.sin(az)
    y = ranges * np.cos(el) * np.cos(az)
    z = ranges * np.sin(el)

    return x, y, z
```

13.3 车载系统为什么要多做一步姿态补偿

如果设备安装在车上，还要再乘上车体姿态矩阵。因为车不是静止平台，它会：

1. 横滚 Roll。
2. 俯仰 Pitch。
3. 航向 Yaw。
4. 随时间移动位置。

所以更完整的空间关系是：

$$\mathbf{p}_{\text{world}} = \mathbf{t}_{\text{gps}} + \mathbf{R}_{\text{vehicle}} (\mathbf{t}_{\text{mount}} + \mathbf{R}_{\text{scanner}} \mathbf{p}_{\text{sensor}})$$

这里可以这样理解：

1. $\mathbf{p}_{\text{sensor}}$ 是传感器本体坐标下的点。
2. $\mathbf{R}_{\text{scanner}}$ 是扫描头姿态。
3. $\mathbf{t}_{\text{mount}}$ 是安装偏置。
4. $\mathbf{R}_{\text{vehicle}}$ 是车体姿态。
5. $\mathbf{t}_{\text{gps}}$ 是车辆位置。

如果你漏掉其中任何一个量，移动扫描点云都会产生系统性偏差。

13.4 为什么还要做网格化

因为 UI 和算法不喜欢直接处理一大堆离散射线点。很多时候你需要把点云落到规则网格上。

例如三维体素索引可以写成：

$$i = \left\lfloor \frac{x - x_0}{\Delta x} \right\rfloor, \quad j = \left\lfloor \frac{y - y_0}{\Delta y} \right\rfloor, \quad k = \left\lfloor \frac{z - z_0}{\Delta z} \right\rfloor$$

然后把同一个体素内的多个点做统计，例如：

- 1. 平均值。
- 2. 最大值。
- 3. 加权平均。
- 4. 命中次数。

13.5 不同页面实际吃什么数据

这是很多项目早期架构里最容易混乱的地方。建议你一开始就把页面和数据层对应清楚。

页面	主要输入	刷新节奏	典型显示
时间-高度主图	L1.5 attenuated backscatter	1 到 10 秒	curtain plot
RHI/PPI 剖面	L2 extinction / PM	每个扫描周期	扇区热力图
3D 页面	L3 voxel / point cloud	5 到 30 秒	体素、切片、热点
告警页面	热点事件表	实时	阈值告警、位置、等级
质量控制页面	L0 / L1 / SNR / energy	实时或分钟级	原始信号和QA图

13.6 一个热点事件最终长什么样

平台层真正喜欢消费的不是整条回波，而是一条结构化事件记录，例如：

```
{
  "event_id": "dust_20260527_102315_001",
  "timestamp": "2026-05-27T10:23:15Z",
  "type": "dust_hotspot",
  "center_enu_m": [85.0, 130.0, 22.5],
  "peak_pm25_ugm3": 186.0,
  "mean_extinction_km_1": 0.42,
  "area_m2": 950.0,
  "confidence": 0.91,
  "recommended_action": "spray"
}
```

你可以看到，这已经不是科研数组，而是业务系统真正能执行动作的结构化对象。

13.7 从事件到喷雾联动

当热点事件已经有了质心坐标，后面通常就是：

- 1. 把 ENU 坐标换成喷雾设备参考坐标。
- 2. 计算目标方位角和俯仰角。
- 3. 判断是否在喷雾炮机械行程范围内。

4. 发送控制指令。
5. 继续观测联动后的压尘效果。

所以从软件视角看，一条完整的数据链并不是“画图结束”，而是：

原始回波数组 -> 预处理 -> 反演 -> PM 估算 -> 空间定位 -> 热点事件 -> 联动控制 -> 效果评估

14. 软件到底应该做成什么样，小白先建立正确的 UI 预期

很多人一说到激光雷达 UI，就先想到炫酷 3D 点云。对颗粒物系统来说，这其实不是第一优先级。

真正高频、好用、工程上有效的界面通常是：

1. 时间-高度热力图。
2. 距离-高度剖面图。
3. 地图上的扫描扇区叠加。
4. 当前廓线曲线。
5. 告警列表和设备状态栏。

3D 当然也重要，但通常是第二屏或分析屏，而不是唯一主屏。

14.1 你应该从公开软件界面里学什么

主要学 3 件事：

1. 信息组织方式。
2. 图层和质量控制怎么展示。
3. 操作员最常盯的指标到底是什么。

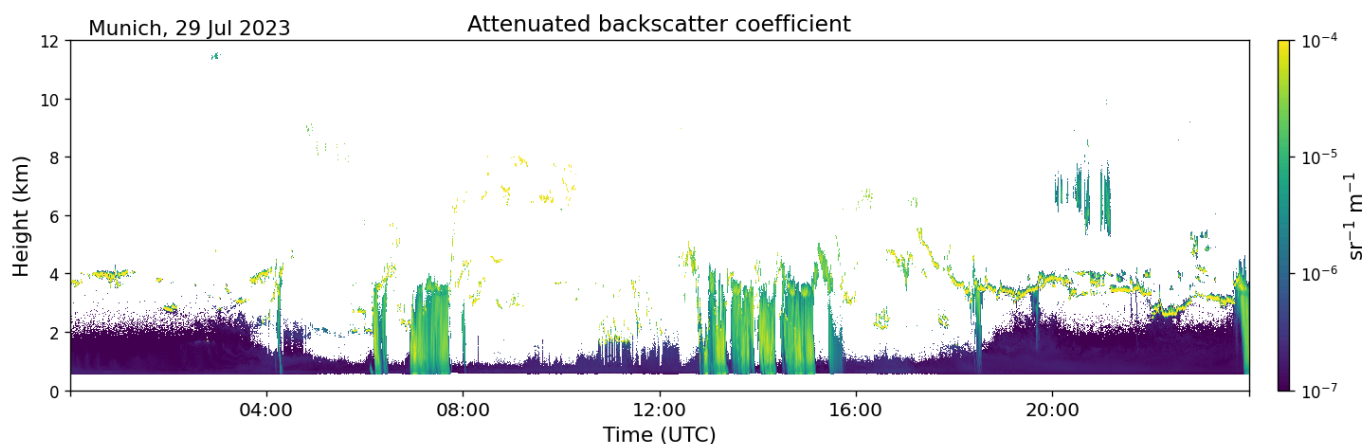
下面这些 UI 图都来自公开资料，已经放进本项目的 assets 目录，可以直接在 Markdown 中查看。

14.2 CloudnetPy / Cloudnet 风格：科研型 quicklook

来源：[CloudnetPy quickstart](#)

这类界面最适合你理解“时间-高度 curtain plot 才是大气遥感系统的主战场”。

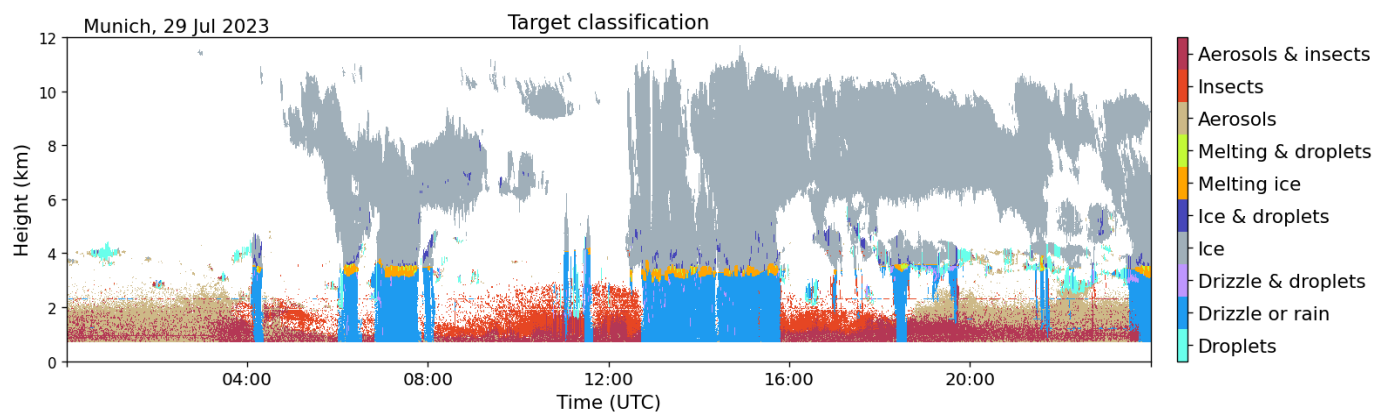
典型 LiDAR attenuated backscatter:



怎么看：

1. 横轴通常是时间。
2. 纵轴通常是高度。
3. 颜色表示后向散射强弱。
4. 一整天的污染层抬升、云底变化、低层气溶胶累积都能直接看出来。

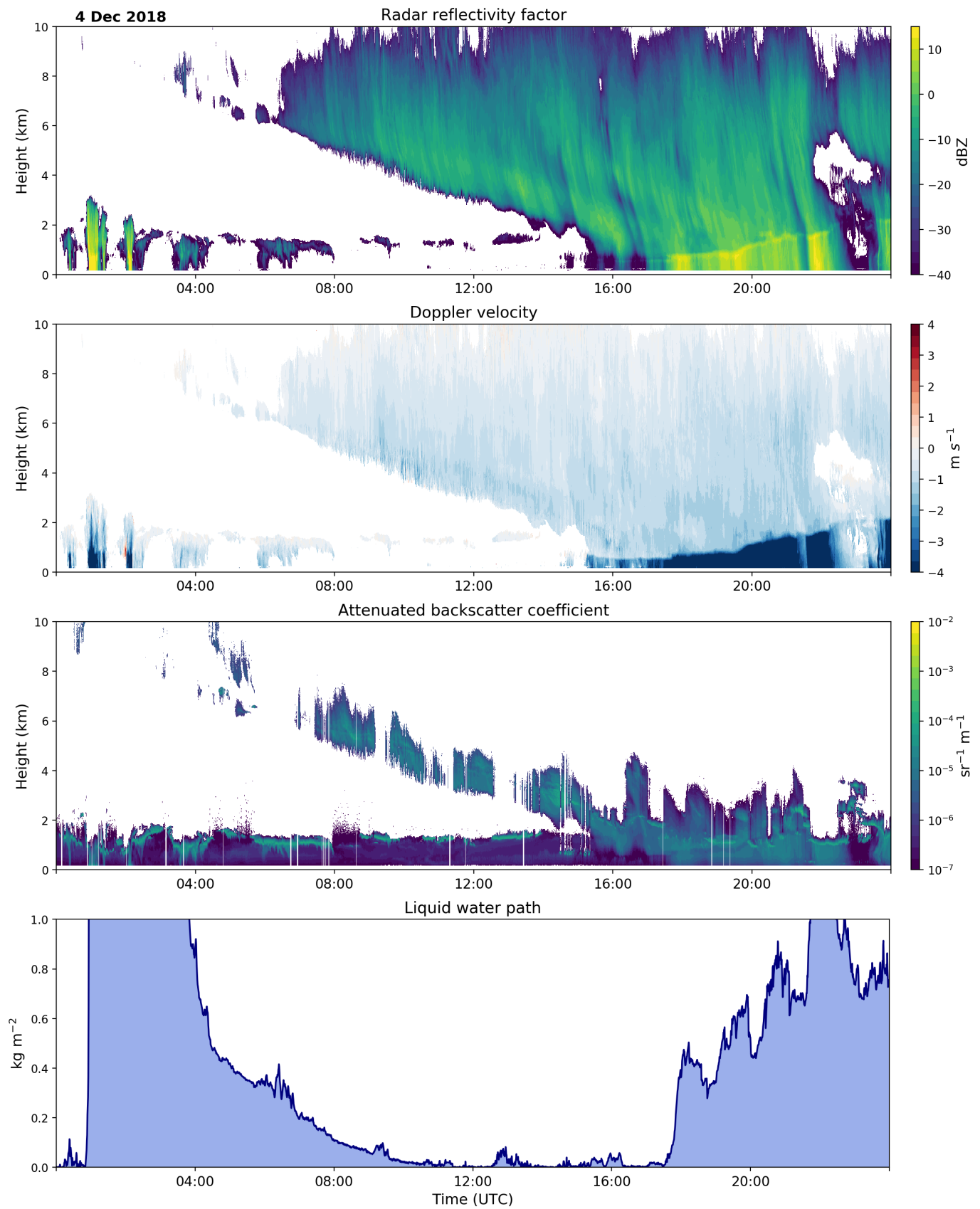
目标分类图：



怎么看：

1. 不同颜色不是“浓淡”，而是不同目标类型。
2. 这类图特别适合未来加入云、雾、雨、昆虫、沙尘等质量控制标签。
3. 对颗粒物系统来说，它能启发你做“有效数据”和“无效数据”的分层展示。

多仪器联合 quicklook：



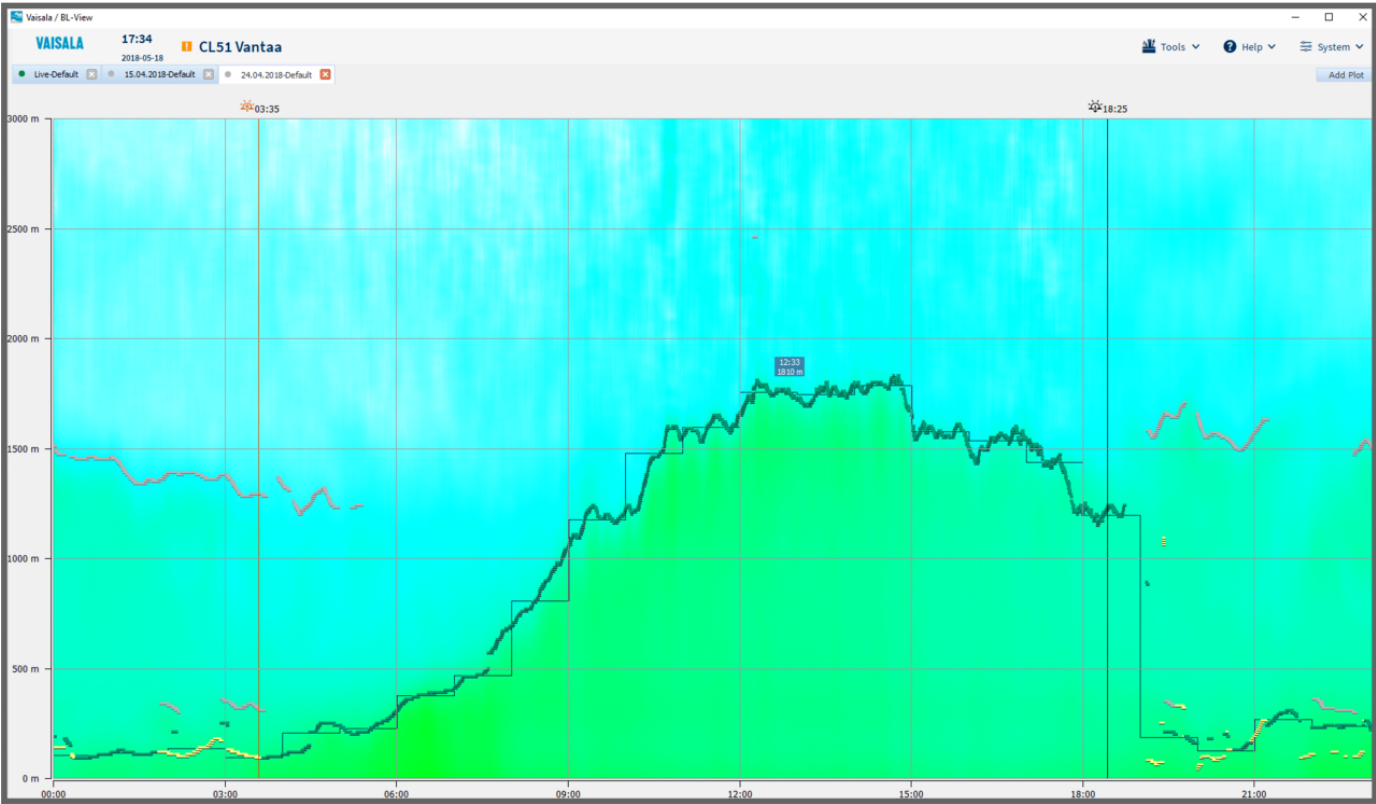
怎么看：

1. 同一时间轴上放多个产品，判断效率会高很多。
2. 平台不应该只有一张图，而应该让用户能同时看回波、气象、质量标志和算法结果。

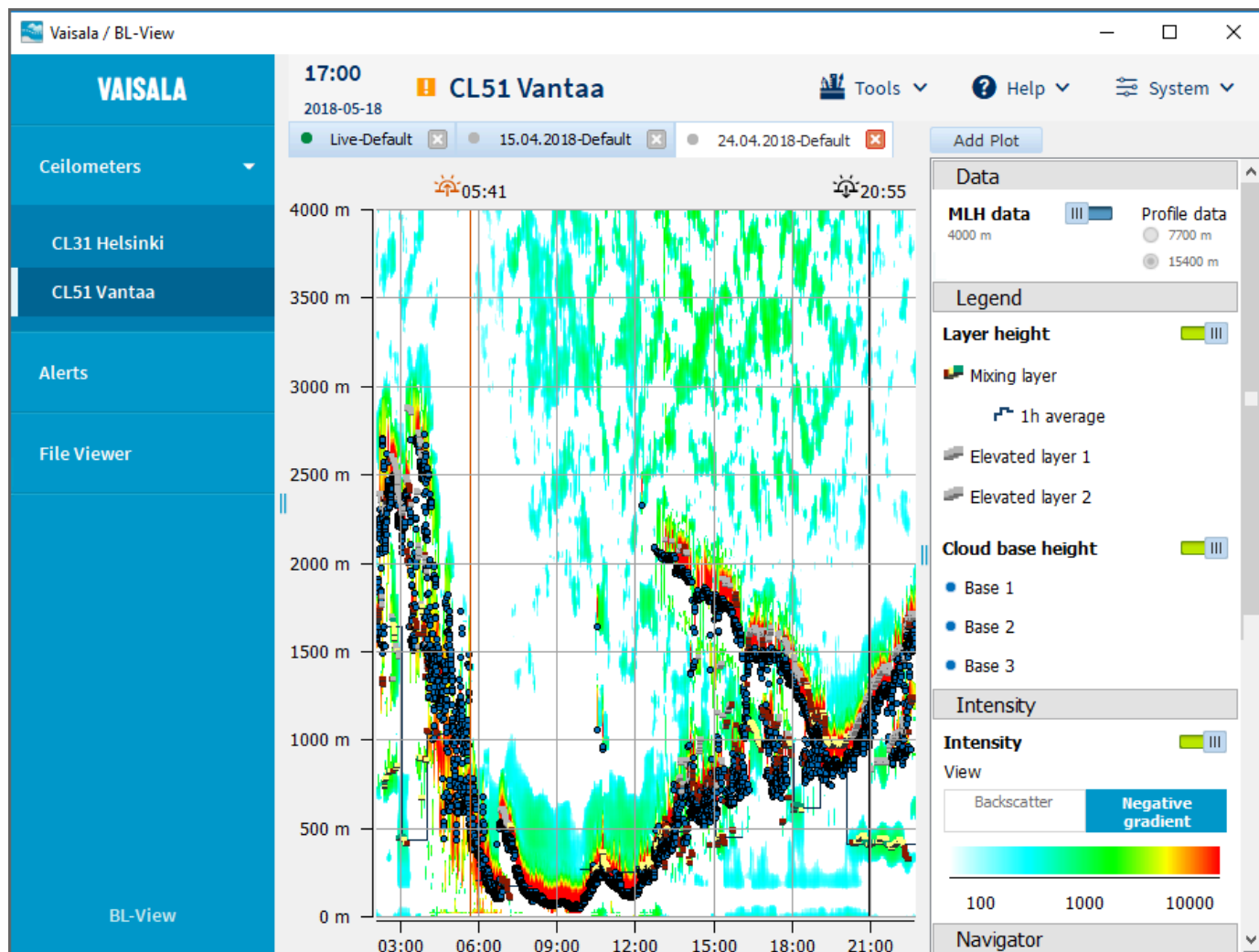
14.3 Vaisala BL-View 风格：运维值守型 UI

来源：[Vaisala BL-View 文档](#)

BL-View 主界面：



归档图界面：



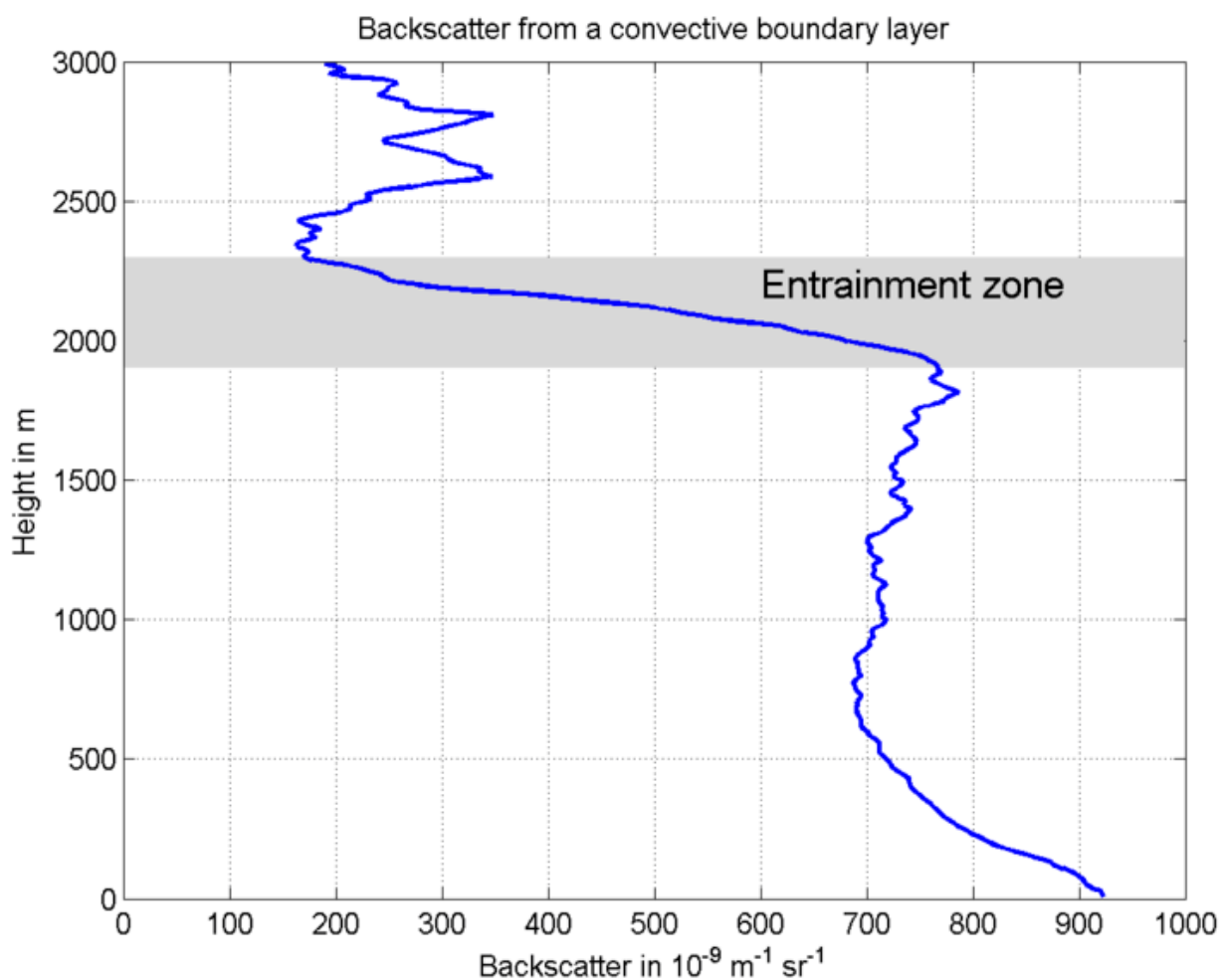
这两张图给你的启发：

1. 顶部状态栏非常重要，不能只顾画图，不顾设备状态。
2. 业务系统必须同时支持实时查看和历史回放。
3. 告警、数据延迟、设备健康度应该一直可见。

14.4 Vaisala CL61 风格：颗粒物业务最接近的显示形态

来源：[Vaisala CL61 文档](#)

CL61 后向散射图：



为什么这张图特别值得参考：

1. 它和颗粒物遥感的主图形态几乎一致。
2. 颜色直接表达强弱，非常适合业务用户。
3. 上面可以继续叠加云底、污染层顶、边界层高度、质量标志线。

14.5 如果做自己的系统，建议至少有 4 个页面

页面 1：实时总览

应该包含：

1. 站点和设备状态栏。
2. 当前风速风向、湿度、温度。
3. 时间-高度热力图。
4. 当前时刻垂直廓线。
5. 当前告警列表。

页面 2：扫描剖面

应该包含：

1. RHI 距离-高度剖面。
2. PPI 扫描扇区图。

3. 产品切换：RCS、消光、PM2.5、PM10、depol。
4. 风场和污染源叠加。

页面 3：三维空间页面

应该包含：

1. 本地底图或三维地形。
2. ENU 点云或体素。
3. 热点质心位置。
4. 时间轴播放控件。
5. 地面站和喷雾设备位置。

页面 4：质量控制与标定页面

应该包含：

1. 原始信号。
2. 背景值和发射能量趋势。
3. SNR 与异常标志。
4. 地面 PM 对比散点图。
5. 模型漂移和误差统计。

14.6 如果最后软件要用 Qt + OpenGL，这几类公开界面最值得参考

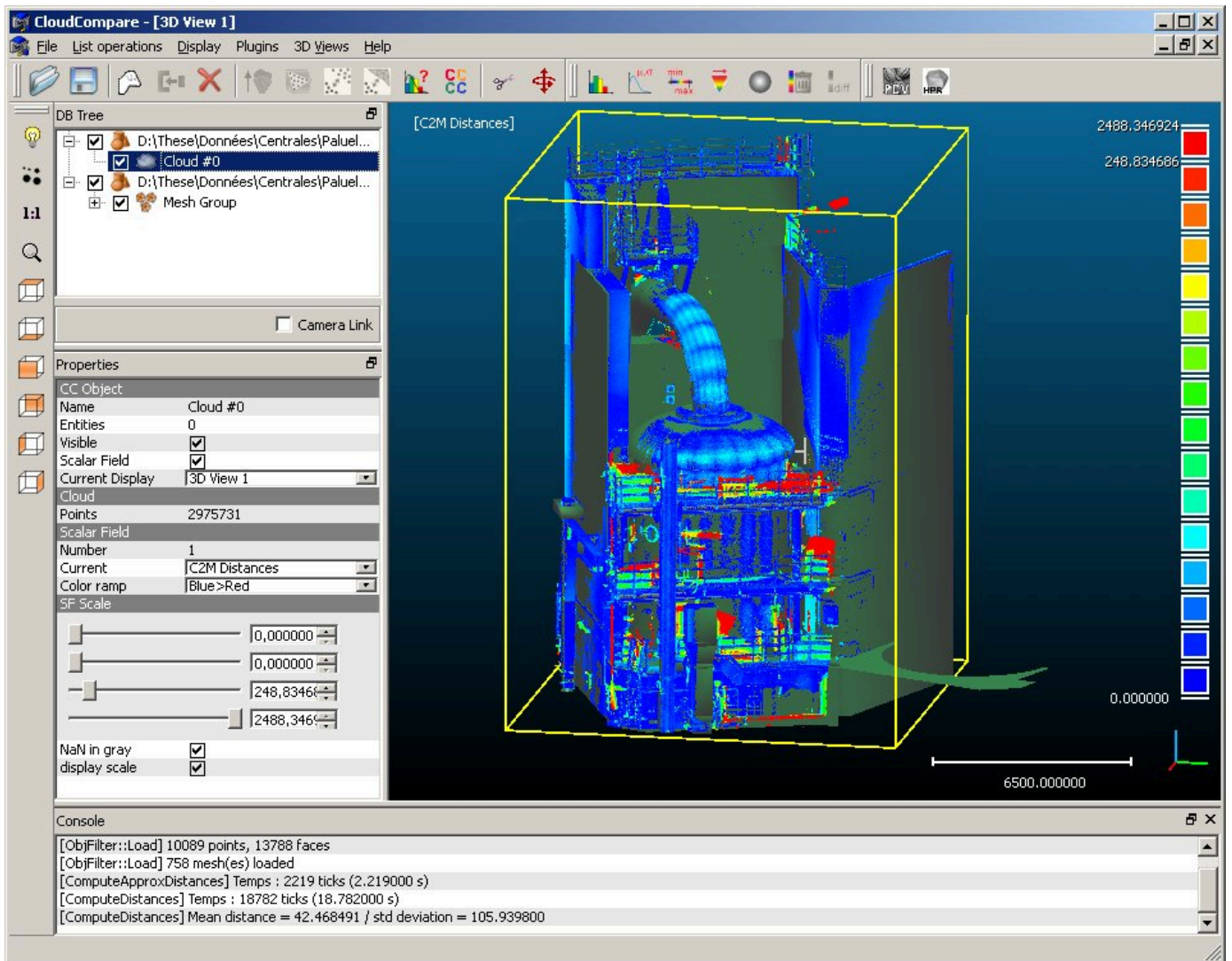
你这次明确提到“最终做成 Qt + OpenGL 的软件”，那参考图就不能只看气象 quicklook 了，还要看真正的桌面软件形态。

最值得参考的是下面 3 类：

1. CloudCompare 这种“左侧对象树 + 中间 OpenGL 三维视图 + 底部日志”的三维分析界面。
2. QGIS 这种“左侧图层树 + 中间地图画布 + 右侧属性面板”的 GIS 业务界面。
3. 把两者结合起来，做成“实时监控主屏偏 2D，分析页面偏 3D”的工业上位机。

如果你现在看到的是 Markdown 源码编辑区，而不是预览面板，那么下面这些图片不会像 Word 那样直接展开，这是正常现象。要看渲染后的图片，请打开 Markdown 预览。

参考图 1: CloudCompare 主界面



原图链接: [CloudCompare 主界面快照](#)

这张图最值得学的不是配色，而是结构：

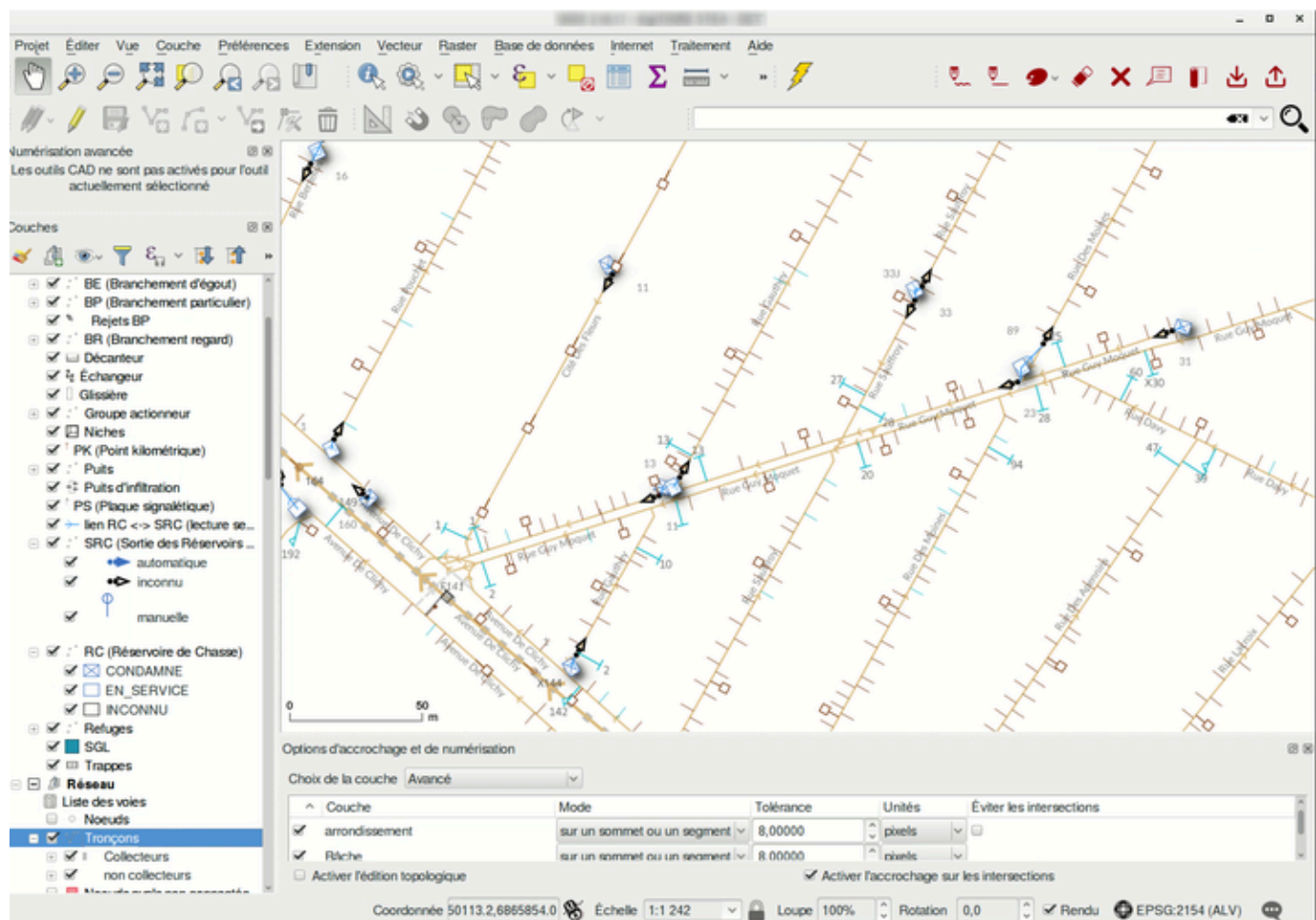
1. 左侧是对象树，适合放扫描任务、图层、热点、设备和历史结果。
2. 中间是 OpenGL 主视口，适合放 ENU 点云、体素、PPI 扇面、RHI 剖面 and 热点质心。
3. 右侧色标告诉你当前颜色到底代表什么物理量，这对 PM、消光、SNR 特别重要。
4. 底部日志区非常实用，适合放设备连接状态、数据延迟、算法异常和控制回执。

如果把它翻译成你的项目，就是：

一定要把“图层树”“主视口”“色标”“日志/状态栏”做成固定骨架，而不是只有一块大画布。

来源：CloudCompare 官方介绍页。官方页面明确写明 CloudCompare 依赖 Qt 和 OpenGL，这张图很适合拿来理解 Qt 桌面软件的典型布局。

参考图 2：QGIS 风格的业务应用界面



原图链接：[QGIS 业务应用界面](#)

这张图更接近“工业上位机”的日常工作状态，因为它不是只展示结果，而是把工具、图层、主画布和底部操作区全部放在一个窗口里。

你重点看 4 个位置：

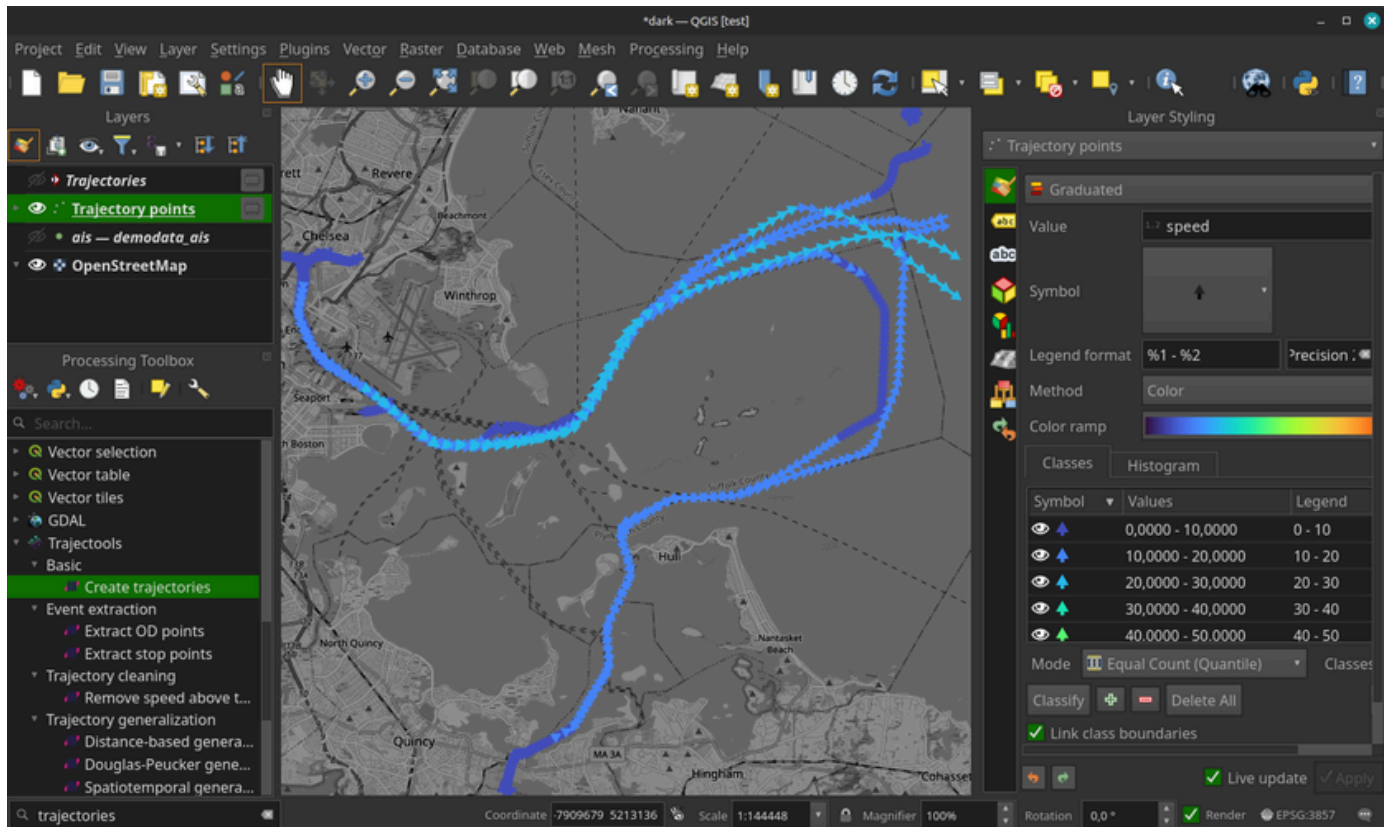
1. 左侧图层树：适合放底图、道路、工地边界、喷雾炮、雷达站位、热点区域。
2. 中间地图区：适合放 PPI 扫描扇区、热点多边形、风向箭头、历史轨迹。
3. 顶部工具栏：适合放开始扫描、暂停、回放、阈值切换、联动开关。
4. 底部参数区：适合放当前目标角、选中区域/属性、设备控制参数。

对你的系统来说，这张图最有启发的一点是：

真正高频使用的主屏通常不是纯 3D，而是“地图 + 图层 + 参数区 + 状态区”的组合界面。

来源：QGIS Hub 截图页，CC0。

参考图 3：QGIS 风格的地图主屏 + 侧边样式面板



原图链接：[QGIS 深色地图主屏](#)

这张图适合拿来理解“分析屏”应该怎么布局：

1. 左边可以是事件树、历史任务树、算法工具箱。
2. 中间是主地图或主剖面区。
3. 右边是样式和阈值面板，适合放颜色表、量程、阈值、图层透明度。

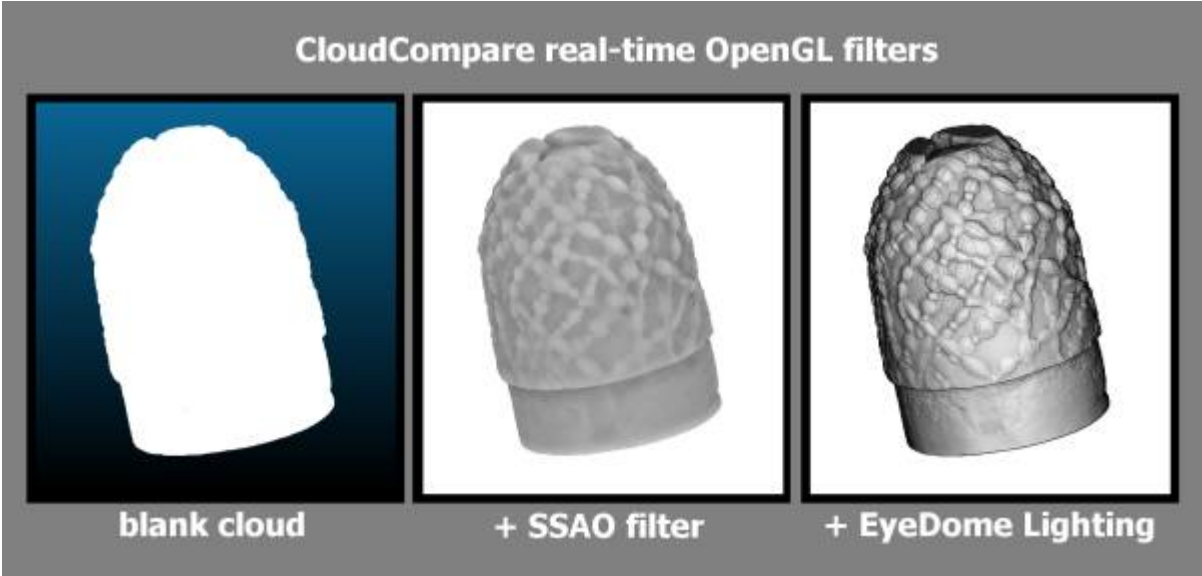
这对颗粒物系统特别重要，因为同一份数据常常要切换到不同显示模式：

1. RCS。
2. 消光系数。
3. PM2.5。
4. PM10。
5. SNR。
6. 告警 mask。

如果没有右侧这种随时可调的样式面板，值班人员很难快速把图调到“看得出问题”的状态。

来源：QGIS Hub 截图页，CC0。

参考图 4：OpenGL 实时渲染效果为什么重要



原图链接：[CloudCompare OpenGL filters](#)

这张图虽然不是完整软件界面，但它很适合解释一个工程事实：

- 1. 同一份三维数据，渲染方式不同，视觉可读性会差很多。
- 2. 如果没有基本的阴影、边缘增强或深度感，三维点云会很平，值班人员不容易看清结构。
- 3. 所以 OpenGL 在这套软件里不只是“为了炫酷”，而是为了把三维结构看清楚。

来源：CloudCompare 官方介绍页中的 OpenGL filters 示例图。

14.7 最推荐的落地路线：Qt Widgets 做外壳，OpenGL 做高性能视图内核

如果你的目标是工业上位机，而不是做一个偏展示的概念 Demo，我更推荐：

- 1. Qt Widgets 负责主窗口、菜单、停靠面板、表格、参数页、设备状态页。
- 2. QOpenGLWidget 负责时间高度图、PPI、RHI、地图叠加和三维场景。
- 3. 曲线和表格继续用 Qt 原生控件或成熟绘图库，不要强行全都塞进 3D 引擎。

这样选的原因很现实：

- 1. QMainWindow + QDockWidget 很适合做工业软件常见的多面板布局。
- 2. OpenGL 只处理真正需要高刷新的图形层，性能和维护成本更平衡。
- 3. 参数配置、日志、设备树、告警表这些内容，用 Widgets 比 QML 或纯 OpenGL 更稳。

如果用一句话概括这条路线，就是：

外壳用 Qt 桌面软件的成熟能力，重图形区域再交给 OpenGL，不要把所有事情都变成三维渲染问题。

14.8 软件模块建议怎么拆

下面这张表是比较适合 MVP 到工程版演进的拆法：

模块	推荐技术	作用
主窗口 AppShell	QMainWindow + QDockWidget + QSplitter	组织页面、停靠面板、工具栏、状态栏
数据接入层	QTcpSocket / QUdpSocket / QSerialPort / QThread	接 LiDAR、气象站、GPS、IMU、喷雾炮控制器

模块	推荐技术	作用
算法调度层	QObject worker + 线程池	背景扣除、RCS、Klett/Fernald、PM 标定、热点检测
产品缓存层	环形缓冲区 + 快照对象	给 UI 提供稳定的 L1/L2/L3 产品快照
2D 图层视图	QOpenGLWidget	时间高度图、PPI、RHI、热力图、地图叠加
3D 视图	QOpenGLWidget	ENU 点云、体素、热点质心、喷雾方向线
曲线与表格	QTableView + QTreeView + 绘图库	当前廓线、设备状态、告警列表、误差统计
回放与归档	SQLite / 文件索引 / 时间轴控件	历史查询、事件回放、前后对比
联动控制层	命令服务 + 状态回执	向喷雾炮、云台、继电器发送控制指令

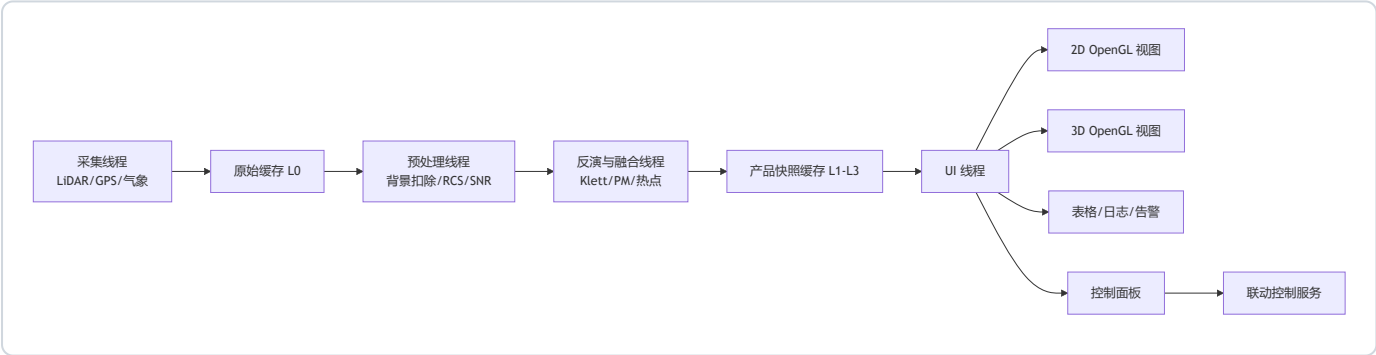
你可以把它理解成 3 层：

- 1. 下层负责收数据和发命令。
- 2. 中层负责把原始数据变成产品。
- 3. 上层负责把产品画出来，并让人能操作。

14.9 线程和数据流一定要这样设计，UI 才不会卡死

Qt + OpenGL 软件最容易犯的错，就是一边采数据、一边算算法、一边在 UI 线程里直接画原始数组。这样数据一大，界面就会卡。

更合理的结构应该是：



你要特别记住 4 条工程规则：

- 1. QOpenGLWidget 的真正绘制发生在 UI 线程，不要在工作线程里直接碰 OpenGL 上下文。
- 2. 算法线程只生产“产品快照”，不要直接操作界面控件。
- 3. UI 线程只消费最新快照，不回头扫大批历史原始数据。
- 4. 采集、算法、UI 之间尽量用 signal/slot 的队列连接或线程安全队列隔开。

也就是说：

算法线程负责算，UI 线程负责看，控制线程负责发命令，三者不要混在一起。

14.10 每一种图在 OpenGL 里分别怎么画

视图 1：时间-高度图

这个视图最适合做成一张持续刷新的二维纹理：

1. 横轴是时间列。
2. 纵轴是距离或高度 bin。
3. 每来一条新 profile，就更新一列像素。
4. 颜色映射由 shader 或颜色查找表完成。

工程上最常用的做法是：

1. CPU 先把当前产品整理成浮点矩阵。
2. 再上传成 OpenGL texture。
3. 最后由 fragment shader 按色标显示。

这样做的好处是，哪怕矩阵比较大，滚动显示也比用普通 QWidget 一格一格画快得多。

视图 2：PPI 和 RHI

PPI 和 RHI 有两条实现路线：

1. 简单路线：先在 CPU 上把极坐标重采样到规则网格，再当作二维纹理画出来。
2. 进阶路线：直接在 GPU 上按射线和距离插值，实时生成扇形或剖面图层。

如果你现在这个项目，我建议先走简单路线，因为：

1. 更容易调试。
2. 算法和显示更容易对齐。
3. 出问题时更容易检查是哪一步错了。

等到 MVP 稳定以后，再考虑把极坐标插值搬到 GPU。

视图 3：地图叠加层

地图页不要直接把所有东西都做成三维。更实用的做法是：

1. 底图作为纹理或瓦片图层。
2. 热点多边形、扫描扇区、设备图标、风向箭头作为叠加图层。
3. 选中某个热点后，在右侧面板显示区域面积、均值、峰值和持续时间。

这类图层最关键的不是炫酷，而是坐标准确。你必须把下面这些对象统一到同一坐标系里：

1. 雷达位置。
2. GPS/IMU 姿态。
3. ENU 结果点。
4. 底图坐标。
5. 喷雾炮目标角。

视图 4：三维点云和体素场

三维视图建议只承担 4 件事：

1. 看热点空间位置。
2. 看喷雾方向和热点是否对准。
3. 看扫描覆盖范围。
4. 做事后分析和回放。

实现时可以这样理解：

1. 点云模式用 VBO 存点坐标和颜色。
2. 体素模式用 instancing 或立方体批量绘制。
3. 色标统一由 PM、消光或 backscatter 对应的查找表控制。
4. 鼠标点击用 picking 或射线相交，拿到选中点或选中体素。

视图 5：当前廓线和质量控制曲线

这一类图不一定非要 OpenGL。更现实的选择是：

1. 用普通 2D 绘图库画当前 profile。
2. 用表格控件画告警和日志。
3. 让 OpenGL 重点负责热力图、地图和三维场景。

这样分工更稳，也更容易维护。

14.11 真正做工程时，页面建议这样拆

如果最后软件要能上线值守，我建议页面按“主屏看态势，副屏做分析”来拆：

1. 实时总览页：时间高度图、当前风场、告警列表、设备状态。
2. 扫描页：PPI、RHI、扇区热力图、扫描参数。
3. 地图联动页：底图、热点区、喷雾炮方向、联动按钮。
4. 三维分析页：ENU 点云、体素、热点轨迹、时间轴回放。
5. 质量控制页：原始信号、能量趋势、背景、SNR、标定散点图。

这样拆的好处是：

1. 值班人员有主屏可看。
2. 算法人员有分析页可查。
3. 运维人员有质量页可排错。
4. 领导或客户演示时也有一页看起来足够直观。

14.12 一个现实可落地的 Qt + OpenGL 开发顺序

如果你们现在准备开始做软件，我建议按下面顺序推进，而不是一开始就做全功能三维系统：

1. 第一步先搭 Qt 主窗口、菜单、状态栏和停靠面板。
2. 第二步先做时间高度图和当前廓线，这样最容易尽快看到成果。
3. 第三步再做 PPI/RHI 和地图叠加。
4. 第四步补三维 ENU 场景和热点选中。
5. 第五步再接历史回放、告警管理和喷雾联动。

这样推进的原因很简单：

1. 最先有业务价值的是 quicklook、剖面 and 地图页。
 2. 三维页很重要，但通常不是 MVP 最先证明价值的页面。
 3. 如果一开始就把大量时间砸在 3D 特效上，反而容易把真正关键的数据链和联动控制拖慢。
-

15. 一个最现实的工程闭环是什么样

如果把“发现扬尘热点并自动处置”作为目标，完整链条可以是：

1. 固定式 LiDAR 连续扫描工地区域。
2. 算法识别高消光或高 PM 估算区域。
3. 系统把该区域转成 ENU 或地理坐标。
4. 平台判断热点是否持续存在、是否超过阈值。
5. 如果满足条件，向喷雾炮发送目标方位角和俯仰角。
6. 喷雾炮执行定向喷雾。
7. LiDAR 继续观测处理后的变化，评估压尘效果。

这个闭环里最重要的不是某一个公式，而是下面 3 个点：

1. 热点识别要稳，不能乱报。
 2. 坐标映射要准，不然喷错方向。
 3. UI 要能让值班人员一眼看懂“为什么报、报在哪里、有没有压下去”。
-

16. 两周 MVP 应该怎么排，不要一开始就铺太大

第 1 周：先把数据链跑通

目标：

1. 打通原始采集或样例数据导入。
2. 完成背景扣除、RCS、基础质量控制。
3. 做出时间-高度图和单帧廓线图。
4. 接一个地面 PM 参考站。

第 1 周做到这一步，其实已经能演示“设备能看见颗粒物结构变化”。

第 2 周：再把业务闭环补上

目标：

1. 加入 Klett/Fernald 的简化版 L2 产品。
2. 做湿度修正和简单 PM 标定。
3. 把扫描数据转到 ENU 坐标。
4. 在地图或 3D 页面上画出热点。
5. 输出告警和喷雾炮控制目标点。

两周 MVP 的核心不是追求“科研终极精度”，而是证明下面这件事成立：

这套系统能够稳定发现工地或道路上的粉尘热点，并在软件上看见、定位、记录、联动处置。

17. 小白最容易掉进去的坑

1. 以为回波强就一定等于 PM 高。

2. 以为只要有激光器就能做系统。
3. 以为 3D 页面一定比 2D 页面更重要。
4. 以为一开始就要追求气体 DIAL。
5. 以为不需要地面参考仪也能做绝对定量。
6. 以为不做 overlap 校正也没关系。
7. 以为湿度影响可以忽略。
8. 以为算法准确就等于工程可用，忽略状态监控和质量控制。

18. 你现在应该先记住哪几句话

如果今天只能带走 6 句话，那就记住这 6 句：

1. 大气颗粒物激光雷达测到的首先是光学回波，不是直接称重。
2. LiDAR 方程里最重要的是“这一层会不会把光打回来”和“整条路径把光吃掉了多少”。
3. 单波长弹性 LiDAR 不能凭一条曲线把所有未知量直接解出来，所以需要假设和标定。
4. 工地扬尘第一版最现实的路线是弹性 Mie 后向散射，不是 DIAL 气体方案。
5. 真正高频使用的 UI 往往是时间-高度图、剖面图、地图扇区图，而不是只有炫酷 3D。
6. 工程价值来自“发现热点、准确定位、闭环联动、持续评估”，不是只生成一张漂亮图片。

19. 一个完整算例：从原始回波到热点告警

前面讲了很多概念和算法，现在用一组简化的假数据，把整条链路真正走一遍。你可以把这一节当成“把前面所有内容串起来的练习题”。

这组数据不是某台真实设备的标定结果，所以它不用于科研发表，但非常适合建立工程直觉。

19.1 先设定一个具体场景

假设我们有一台固定式颗粒物 LiDAR，装在工地高点平台上：

1. 设备安装高度为 18 m。
2. 当前在看一个固定方位附近的低仰角扫描。
3. 距离分辨率为 30 m。
4. 我们先只看一条射线上的 6 个 range bin。
5. 然后再把这条射线扩展成一个小扫描扇面，最后提取热点。

为了便于说明，先定义 6 个距离 bin：

$$R = [30, 60, 90, 120, 150, 180] \text{ m}$$

原始 photon counts 假设如下：

$$P_{\text{raw}} = [34, 38, 55, 110, 82, 45]$$

同时假设：

1. 远端背景均值 $B = 20$ counts。

2. 这一帧的激光单脉冲能量 $E = 2.0 \text{ mJ}$ 。

3. overlap 曲线在近距离不完整：

$$O(R) = [0.40, 0.70, 0.90, 1.00, 1.00, 1.00]$$

19.2 第一步：背景扣除

背景扣除公式是：

$$P_1(R) = P_{\text{raw}}(R) - B$$

带入这组数据后：

$$P_1 = [14, 18, 35, 90, 62, 25]$$

这里最值得注意的是：

1. 原始 34 counts 并不表示有 34 counts 都来自目标。
2. 其中有 20 counts 只是背景底噪。
3. 真正属于目标回波的只有扣背景之后的部分。

你可以把这一步理解成：

先把房间里的环境光关掉，再看手电筒真正照到了什么。

19.3 第二步：发射能量归一化

能量归一化公式是：

$$P_2(R) = \frac{P_1(R)}{E}$$

带入 $E = 2.0 \text{ mJ}$ 后：

$$P_2 = [7.00, 9.00, 17.50, 45.00, 31.00, 12.50]$$

单位你可以先理解成“每毫焦对应的计数”。

为什么这一步不能省：

1. 因为不同脉冲的发射能量会抖动。
2. 不归一化的话，两帧之间的差别不一定来自空气变化，也可能只是激光器输出变强或变弱。

19.4 第三步：overlap 校正

现在把近距离重叠不完全的问题补回来：

$$P_3(R) = \frac{P_2(R)}{O(R)}$$

计算后得到：

$$P_3 \approx [17.50, 12.86, 19.44, 45.00, 31.00, 12.50]$$

这一步有一个很重要的现象：

- 1. 30 m 处本来只有 7.00。
- 2. 但因为 overlap 只有 0.40，校正后变成了 17.50。

这说明近距离如果不做 overlap 校正，近端污染会被明显低估。

19.5 第四步：RCS 距离平方校正

距离平方校正公式：

$$RCS(R) = P_3(R)R^2$$

把每个距离 bin 算出来：

距离 R (m)	原始计数 P _{raw}	扣背景后 P1	能量归一化 P2	overlap 校正后 P3	RCS
30	34	14	7.00	17.50	15750
60	38	18	9.00	12.86	46296
90	55	35	17.50	19.44	157464
120	110	90	45.00	45.00	648000
150	82	62	31.00	31.00	697500
180	45	25	12.50	12.50	405000

从这个表里你可以立刻看到两件事：

- 1. 120 m 到 150 m 之间是这条射线里最显著的高回波区。
- 2. 150 m 的 RCS 比 120 m 还高，说明那里可能仍然存在明显颗粒物结构。

但注意，这里还不能直接说“150 m 的 PM 一定最高”，因为传播衰减还没有真正被反演掉。

19.6 第五步：给 Klett/Fernald 反演准备边界条件

现在开始做反演。我们先给出一个工程上常见的简化假设：

- 1. 取 lidar ratio $S = 45$ sr。
- 2. 取最远端 180 m 作为参考 bin。
- 3. 假设参考端消光系数：

$$\alpha_{\text{ref}} = 0.18 \text{ km}^{-1}$$

这相当于告诉算法：

最远端那一格我先给你一个可接受的起点，你从这里往回推。

19.7 第六步：做一个简化版 Klett 反演

为了便于教学，我们直接给出这组假数据经过简化递推后的结果。你可以把它理解成“程序已经根据 RCS、边界条件和 lidar ratio 算好的输出”。

距离 R (m)	backscatter β (km ⁻¹ sr ⁻¹)	extinction $\alpha = S\beta$ (km ⁻¹)
30	0.0019	0.086
60	0.0022	0.099
90	0.0030	0.135
120	0.0063	0.284
150	0.0051	0.230
180	0.0040	0.180

这里的结果比单看 RCS 更有物理意义，因为它试图把双程传播衰减的影响部分剥离掉。

从结果上看：

- 1. 120 m 的消光系数最高。
- 2. 150 m 也仍然偏高。
- 3. 说明这条射线上真正的主污染区大致位于 120 m 到 150 m。

这就是为什么：

你不能只看 RCS 或颜色深浅，而必须通过反演去判断真正的光学浓度结构。

19.8 第七步：加入湿度修正

现在假设这条路径上的相对湿度随距离略有变化：

$$RH = [72, 74, 76, 78, 80, 82] \%$$

并且取吸湿增长经验参数：

$$\gamma = 0.45$$

湿度修正公式：

$$\alpha_{\text{dry}}(R) = \alpha(R) \cdot \left[1 - \frac{RH(R)}{100}\right]^\gamma$$

计算后得到：

距离 R (m)	原始消光 α	RH (%)	干态消光 α_{dry}
30	0.086	72	0.048
60	0.099	74	0.054
90	0.135	76	0.071
120	0.284	78	0.144
150	0.230	80	0.112
180	0.180	82	0.083

这个表最值得你注意的点是：

- 1. 所有点的消光都下降了。
- 2. 120 m 仍然是最高值。
- 3. 说明它的高值不只是“湿度太大造成的虚高”，而是真有明显颗粒物结构。

19.9 第八步：把干态消光转成 PM2.5 / PM10

为了教学简单，我们先用一个非常朴素的线性模型：

$$PM_{2.5} = 500 \cdot \alpha_{dry} + 10$$

再假设：

$$PM_{10} = 1.6 \cdot PM_{2.5}$$

注意：这只是教学用经验关系，真实项目里必须用本地标定模型替代。

带入后得到：

距离 R (m)	干态消光 α_{dry}	PM2.5 ($\mu g/m^3$)	PM10 ($\mu g/m^3$)
30	0.048	34.0	54.4
60	0.054	37.0	59.2
90	0.071	45.5	72.8
120	0.144	82.0	131.2
150	0.112	66.0	105.6
180	0.083	51.5	82.4

如果我们把热点阈值设为：

$$PM_{2.5} > 60 \mu g/m^3$$

那么这条射线上会有两个连续高值 bin：

- 1. 120 m。
- 2. 150 m。

到这里为止，你已经从一条原始回波数组，走到了“这条射线上的高污染区在哪里”。

19.10 第九步：把单条射线扩展成一个小扫描扇面

现在我们不只看一条射线，而是假设相邻两个方位角也扫到了类似高值。为了简化说明，只取 4 个热点元点：

点编号	方位角 (deg)	仰角 (deg)	距离 R (m)	PM2.5
A	40	10	120	82
B	40	10	150	66
C	42	10	120	79

点编号	方位角 (deg)	仰角 (deg)	距离 R (m)	PM2.5
D	42	10	150	63

这些点说明：

- 1. 高值区不只出现在一条射线上。
- 2. 它已经在相邻扫描角里形成了一个连续的二维小斑块。
- 3. 这就具备了“提取热点区域”的基础。

19.11 第十步：把热点元转成 ENU 坐标

仍然使用固定式系统的 ENU 公式：

$$\begin{aligned}x &= R \cos(\text{elevation}) \sin(\text{azimuth}) \\y &= R \cos(\text{elevation}) \cos(\text{azimuth}) \\z &= R \sin(\text{elevation})\end{aligned}$$

算出来的近似结果如下：

点编号	ENU x (m)	ENU y (m)	ENU z (m)	权重 PM2.5
A	75.9	90.5	20.8	82
B	94.9	113.2	26.0	66
C	79.0	87.8	20.8	79
D	98.8	109.7	26.0	63

此时这些点已经不是抽象的“第几个 bin”，而是真正的空间点。

19.12 第十一步：计算热点质心

如果用 PM2.5 做权重，那么热点加权质心公式是：

$$x_c = \frac{\sum_i w_i x_i}{\sum_i w_i}, \quad y_c = \frac{\sum_i w_i y_i}{\sum_i w_i}, \quad z_c = \frac{\sum_i w_i z_i}{\sum_i w_i}$$

带入 A、B、C、D 四个点之后，可以得到近似质心：

$$(x_c, y_c, z_c) \approx (86.1, 99.1, 23.2) \text{ m}$$

这意味着什么？

意思是如果你要驱动一个喷雾炮去瞄准这个热点，那么它最值得优先瞄准的位置，不是 A、B、C、D 某一个点，而是这个加权质心附近。

19.13 第十二步：把质心转成控制目标

对于固定式喷雾炮，平台通常还要继续算两个量：

- 1. 水平距离。

2. 控制方位角和俯仰角。

水平距离：

$$R_{\text{horizontal}} = \sqrt{x_c^2 + y_c^2} \approx 131.2 \text{ m}$$

目标方位角近似为：

$$\text{heta} = \arctan\left(\frac{x_c}{y_c}\right) \approx 41^\circ$$

目标仰角近似为：

$$\phi = \arctan\left(\frac{z_c}{R_{\text{horizontal}}}\right) \approx 10^\circ$$

这个结果很合理，因为它正好落在前面 40 到 42 度、10 度仰角那一小片高值区中间。

19.14 第十三步：形成热点事件记录

到平台层，通常会把它整理成一条可消费事件：

```
{
  "event_id": "dust_demo_001",
  "timestamp": "2026-05-27T10:23:15Z",
  "type": "dust_hotspot",
  "center_enu_m": [86.1, 99.1, 23.2],
  "target_azimuth_deg": 41.0,
  "target_elevation_deg": 10.0,
  "peak_pm25_ugm3": 82.0,
  "mean_pm25_ugm3": 72.5,
  "recommended_action": "spray"
}
```

此时这条数据已经非常适合：

1. 给前端画告警卡片。
2. 给地图画一个热点标记。
3. 给喷雾联动模块发控制指令。
4. 给报表系统留痕。

19.15 用一段代码把这个算例串起来

下面是一段非常简化但逻辑完整的示意代码，用来把这一节的算例串起来：

```

import numpy as np

ranges = np.array([30, 60, 90, 120, 150, 180], dtype=float)
raw_counts = np.array([34, 38, 55, 110, 82, 45], dtype=float)
background = 20.0
laser_energy = 2.0
overlap = np.array([0.40, 0.70, 0.90, 1.00, 1.00, 1.00], dtype=float)
rh = np.array([72, 74, 76, 78, 80, 82], dtype=float)

signal_bg = raw_counts - background
signal_norm = signal_bg / laser_energy
signal_overlap = signal_norm / overlap
rcs = signal_overlap * ranges**2

beta = np.array([0.0019, 0.0022, 0.0030, 0.0063, 0.0051, 0.0040])
alpha = 45.0 * beta
alpha_dry = alpha * (1.0 - rh / 100.0) ** 0.45

pm25 = 500.0 * alpha_dry + 10.0
pm10 = 1.6 * pm25

hot_mask = pm25 > 60.0
hot_ranges = ranges[hot_mask]
hot_pm25 = pm25[hot_mask]

print("RCS:", np.round(rcs, 1))
print("alpha:", np.round(alpha, 3))
print("PM2.5:", np.round(pm25, 1))
print("热点距离:", hot_ranges)
print("热点PM2.5:", np.round(hot_pm25, 1))

```

你可以看到，这段代码虽然不长，但它已经完整体现了这条思路：

1. 原始回波先做清洗。
2. 再做 RCS。
3. 再做反演输出光学量。
4. 再做湿度修正。
5. 再做 PM 推断。
6. 最后再做热点判别。

19.16 这个完整算例真正想让你学会什么

如果你把这节真正看懂了，其实就已经建立了一个很重要的工程直觉：

1. 原始回波不是业务结果，它只是起点。
2. 预处理不是锦上添花，而是决定后面会不会全盘跑偏。
3. 反演解决的是“把回波变成光学量”的问题。

4. 湿度修正和地面标定解决的是“把光学量变成 PM”的问题。
5. ENU 映射和热点提取解决的是“把 PM 场变成可行动坐标”的问题。
6. 最终平台真正消费的，不是一条波形，而是一条结构化事件。

如果你愿意，下一步完全可以把这一节再继续升级成：

1. 带真实 Python 可运行脚本的版本。
2. 带图表输出的 notebook 版本。
3. 带 RHI 或 PPI 小矩阵示例的二维版本。

19.17 再进一步：二维 PPI 扫描面小矩阵算例

上面的例子本质上还是“少量热点点点”。现在我们把它再往前推进一步，真正做成一个二维扫描面。

这里选择 PPI 作为例子，因为它更贴近工地和道路场景里最常见的“水平扩散监控”。

PPI 的意思是：

1. 仰角固定。
2. 方位角不断变化。
3. 每个方位角上都有一串距离 bin。

所以 PPI 的数据天然就是一个二维矩阵：

$$PM[azimuth, R]$$

你可以把它理解成：

每一行是一条射线，每一列是这条射线上的一个距离格子，整张表拼起来就是一个扫描扇面。

19.18 先定义一个最小 PPI 小矩阵

假设：

1. 固定仰角为 10° 。
2. 方位角一共扫 3 条： 38° 、 40° 、 42° 。
3. 距离 bin 取 4 个：60 m、90 m、120 m、150 m。
4. 每个格子的 PM2.5 都已经经过前面的整条算法链，也就是已经完成了预处理、反演和湿度修正后的标定输出。

得到的二维 PM2.5 小矩阵如下：

方位角 \ 距离	60 m	90 m	120 m	150 m
38 deg	34	46	78	58
40 deg	37	51	82	66
42 deg	35	49	79	63

读这个表时，你要这样看：

1. 同一行表示同一条扫描射线。
2. 同一列表示相近的距离圈层。
3. 右上区域数值更高，说明热点不是单点，而是一个小片区。

19.19 第一步：做阈值分割

仍然使用前面的业务阈值：

$$PM_{2.5} > 60 \mu g/m^3$$

把矩阵转成二值 mask 后：

方位角 \ 距离	60 m	90 m	120 m	150 m
38 deg	0	0	1	0
40 deg	0	0	1	1
42 deg	0	0	1	1

这里的 1 表示热点候选单元，0 表示非热点单元。

从这个二值表里，你已经能直观看见一件事：

- 1. 高值区主要集中在 120 m 到 150 m。
- 2. 并且跨越了多个相邻方位角。
- 3. 这说明它不是随机噪点，而更像一个连续粉尘斑块。

19.20 第二步：连通域判断这个热点是不是一个整体

对二维扫描面来说，阈值分割之后通常还要做连通域分析。

在这个小例子里，值为 1 的格子共有 5 个：

- 1. 38 deg, 120 m。
- 2. 40 deg, 120 m。
- 3. 40 deg, 150 m。
- 4. 42 deg, 120 m。
- 5. 42 deg, 150 m。

如果采用 8 邻域连通规则，这 5 个格子会被识别为同一个热点区域，而不是 5 个彼此无关的小点。

这一步非常关键，因为平台真正关心的是：

这里是不是存在一个连续污染团，而不是某一个像素偶然超阈值。

19.21 第三步：把二维热点格子转成空间坐标

现在把这 5 个热点格子真正转换到 ENU 坐标。

固定仰角仍然是 10°，转换公式与前面一致：

$$\begin{aligned} x &= R \cos(10^\circ) \sin(\text{azimuth}) \\ y &= R \cos(10^\circ) \cos(\text{azimuth}) \\ z &= R \sin(10^\circ) \end{aligned}$$

算出来的热点格子中心坐标近似如下：

热点格子	PM2.5	x (m)	y (m)	z (m)
38 deg, 120 m	78	72.8	93.1	20.8
40 deg, 120 m	82	76.0	90.5	20.8
42 deg, 120 m	79	79.0	87.8	20.8
40 deg, 150 m	66	94.9	113.2	26.0
42 deg, 150 m	63	98.8	109.7	26.0

到了这里，二维扫描面上的“5 个高值格子” 就已经被转换成了“5 个真实空间点”。

19.22 第四步：估算热点覆盖面积

PPI 里的每个格子并不是正方形，而更接近一个扇形小单元。

对于这个例子：

1. 距离步长是 30 m。
2. 方位角步长是 2 度，约等于 0.0349 rad。
3. 仰角固定为 10 度。

在低仰角条件下，每个格子投影到水平面的面积可以近似写成：

$$A_{\text{cell}}(R) \approx R \times 30 \times 0.0349 \times \cos^2(10^\circ)$$

所以：

1. 120 m 这一圈的单元面积大约是 122 m^2 。
2. 150 m 这一圈的单元面积大约是 152 m^2 。

因为热点里有：

1. 3 个 120 m 单元。
2. 2 个 150 m 单元。

所以总热点面积近似为：

$$3 \times 122 + 2 \times 152 \approx 670 \text{ m}^2$$

这一步的意义在于：

1. 你不只是知道“哪里超标”。
2. 还知道这个热点大概覆盖了多大范围。
3. 这对喷雾持续时间和处置优先级判断很有用。

19.23 第五步：计算二维热点的加权质心

现在用 PM2.5 作为权重，计算这 5 个热点单元的加权质心。

公式和前面一样：

$$x_c = \frac{\sum_i w_i x_i}{\sum_i w_i}, \quad y_c = \frac{\sum_i w_i y_i}{\sum_i w_i}, \quad z_c = \frac{\sum_i w_i z_i}{\sum_i w_i}$$

把这 5 个单元代入之后，可以得到近似结果：

$$(x_c, y_c, z_c) \approx (83.3, 97.9, 22.7) \text{ m}$$

这个质心比前面 4 点小例子的结果略微更靠近近端一点，是合理的，因为这次热点分布里 120 m 的高权重点更多。

19.24 第六步：把二维热点质心转成喷雾目标角

有了质心坐标之后，控制系统常要继续算：

1. 水平距离。
2. 目标方位角。
3. 目标仰角。

水平距离：

$$R_{\text{horizontal}} = \sqrt{x_c^2 + y_c^2} \approx 128.5 \text{ m}$$

目标方位角近似为：

$$\text{az}_{\text{target}} = \arctan\left(\frac{x_c}{y_c}\right) \approx 40.4^\circ$$

目标仰角近似为：

$$\phi_{\text{target}} = \arctan\left(\frac{z_c}{R_{\text{horizontal}}}\right) \approx 10.0^\circ$$

从这个结果你可以看到，二维扫描面算出来的热点目标角，仍然稳定落在 40 度左右、10 度左右，这和我们一开始的人眼直观判断是一致的。

19.25 第七步：如果把这个二维结果画到 UI 上，会长什么样

在 PPI 页面上，它不会表现成 5 个孤立数字，而会更像一个扇形网格中的连续高值区。

你可以把它抽象成下面这样：

```
PPI fixed elevation = 10 deg

azimuth \ range  60m   90m   120m  150m
38 deg          low   low   high   low
40 deg          low   low   high   high
42 deg          low   low   high   high

centroid -> (83.3, 97.9, 22.7) m
target    -> az=40.4 deg, el=10.0 deg
```

真正的前端显示里，通常会同时叠加：

1. 热力图颜色。
2. 阈值轮廓线。
3. 热点质心点。
4. 当前建议喷雾指向。

这时候值班人员看到的就不再是抽象矩阵，而是一张可直接决策的业务图。

19.26 用代码把这个二维算例串起来

下面这段示意代码把二维 PPI 小矩阵的核心步骤连起来：

```
import numpy as np

azimuths = np.array([38.0, 40.0, 42.0], dtype=float)
ranges = np.array([60.0, 90.0, 120.0, 150.0], dtype=float)
elevation = 10.0

pm25 = np.array([
    [34.0, 46.0, 78.0, 58.0],
    [37.0, 51.0, 82.0, 66.0],
    [35.0, 49.0, 79.0, 63.0],
])

mask = pm25 > 60.0
hotspots = []

for iaz, azimuth in enumerate(azimuths):
    az = np.deg2rad(azimuth)
    el = np.deg2rad(elevation)

    for ir, radius in enumerate(ranges):
        if not mask[iaz, ir]:
            continue

        value = pm25[iaz, ir]
        x = radius * np.cos(el) * np.sin(az)
        y = radius * np.cos(el) * np.cos(az)
        z = radius * np.sin(el)
        hotspots.append((value, x, y, z))

weights = np.array([row[0] for row in hotspots])
xyz = np.array([row[1], row[2], row[3]] for row in hotspots)
centroid = (weights[:, None] * xyz).sum(axis=0) / weights.sum()

print("hotspot_count =", len(hotspots))
print("centroid_xyz =", np.round(centroid, 2))
```

这段代码体现的是二维扫描面的标准流程：

- 1. 二维矩阵先做阈值筛选。
- 2. 保留热点单元。
- 3. 把热点单元逐个转成三维坐标。
- 4. 最后再算质心和控制目标。

19.27 这个二维算例真正想让你明白什么

这一节最重要的，不是记住某个数字，而是建立下面这套直觉：

- 1. 一维射线算例解决的是“单条光路怎么看”。
- 2. 二维 PPI 算例解决的是“一个扫描面怎么看”。
- 3. 真正的业务热点，不是单个 bin，而是二维或三维上的连续区域。
- 4. 平台做告警和联动时，核心对象不是波形，而是区域、质心、面积和目标角。

19.28 再补一个垂直剖面例子：RHI 到底怎么看

前面的 PPI 解决的是“热点在平面上扩到哪里去了”。

RHI 解决的则是另外一个非常关键的问题：

这个污染羽流抬升到了多高、层底层顶在哪里、是不是正在往上翻卷。

RHI 的含义是：

- 1. 方位角固定。
- 2. 仰角不断变化。
- 3. 在一个固定方向上把空间剖开，得到一张“距离-高度”剖面图。

对工地扬尘、料场扬尘、烟羽抬升判断来说，RHI 特别有用，因为它能直接告诉你：

- 1. 粉尘是贴地扩散，还是已经被抬到半空。
- 2. 热点的垂直厚度有多大。
- 3. 喷雾是该打近地面，还是该打更高的位置。

19.29 先定义一个最小 RHI 小矩阵

假设：

- 1. 固定方位角为 40 deg。
- 2. 设备安装高度为 18 m。
- 3. 仰角扫 4 条：4 deg、8 deg、12 deg、16 deg。
- 4. 距离 bin 取 4 个：60 m、90 m、120 m、150 m。
- 5. 每个格子的值依然是假设已经经过完整算法链输出的 PM2.5。

于是得到一个二维 RHI 小矩阵：

仰角 \ 距离	60 m	90 m	120 m	150 m
4 deg	28	45	72	65
8 deg	32	58	95	84

仰角 \ 距离	60 m	90 m	120 m	150 m
12 deg	30	55	88	76
16 deg	24	41	62	54

你可以先直接目测这张表：

1. 主要高值集中在 120 m 到 150 m。
2. 高值覆盖了多个仰角。
3. 说明这不是地面一条很薄的带，而是一个向上抬起的垂直污染羽流。

19.30 第一步：做阈值分割

还是沿用前面的业务阈值：

$$PM_{2.5} > 60 \mu g/m^3$$

转成二值 mask 之后：

仰角 \ 距离	60 m	90 m	120 m	150 m
4 deg	0	0	1	1
8 deg	0	0	1	1
12 deg	0	0	1	1
16 deg	0	0	1	0

这个结果说明：

1. 热点主要存在于 120 m 到 150 m 这一带。
2. 从 4 deg 一直延伸到 16 deg。
3. 所以它不是单点，而是一条有厚度的竖向污染剖面。

19.31 第二步：把 RHI 格子转成地距-高度坐标

对固定方位角的 RHI 来说，最常用的不是先转完整三维 ENU，而是先转成剖面坐标：

1. 地面投影距离 s 。
2. 高度 h 。

公式是：

$$s = R \cos(\text{elevation})$$
$$h = h_0 + R \sin(\text{elevation})$$

其中：

1. $h_0 = 18\text{ m}$ 是设备安装高度。
2. s 表示沿这个固定方位方向，离设备有多远。
3. h 表示这个格子真正处在多高的位置。

把所有热点格子换算后，得到近似结果：

热点格子	PM2.5	地距 s (m)	高度 h (m)
4 deg, 120 m	72	119.7	26.4
4 deg, 150 m	65	149.6	28.5
8 deg, 120 m	95	118.8	34.7
8 deg, 150 m	84	148.5	38.9
12 deg, 120 m	88	117.4	42.9
12 deg, 150 m	76	146.7	49.2
16 deg, 120 m	62	115.4	51.1

这张表特别重要，因为到这一步你终于能回答：

1. 污染并不是只在地面附近。
2. 它已经从大约 26 m 一直抬升到 51 m 左右。
3. 这说明这是一个有明显竖向结构的羽流，而不是单纯贴地积尘。

19.32 第三步：估算层底、层顶和剖面厚度

RHI 最常见的一个业务动作，就是从热点区里估算：

1. 层底高度。
2. 层顶高度。
3. 垂直厚度。

对这组数据：

$$h_{\text{bottom}} \approx 26.4 \text{ m}$$

$$h_{\text{top}} \approx 51.1 \text{ m}$$

所以剖面厚度约为：

$$\text{depth}_{\text{vertical}} = h_{\text{top}} - h_{\text{bottom}} \approx 24.7 \text{ m}$$

这一组量在业务里非常有价值，因为它直接回答了：

1. 羽流最高抬到了多高。
2. 近地喷雾能不能覆盖到它。
3. 这个污染区是“薄薄一层”还是“厚厚一团”。

19.33 第四步：计算 RHI 剖面的加权质心

如果用 PM2.5 作为权重，那么在 RHI 平面里我们常先算一个剖面质心：

$$s_c = \frac{\sum_i w_i s_i}{\sum_i w_i}, \quad h_c = \frac{\sum_i w_i h_i}{\sum_i w_i}$$

把上面 7 个热点格子代入之后，可得到近似结果：

$$(s_c, h_c) \approx (130.5, 38.7) \text{ m}$$

这个结果怎么理解：

1. 这条剖面里的主要污染团，平均位于设备前方约 130 m。
2. 它的主质量中心大约在 39 m 高度。
3. 如果你要做垂直剖面的人工研判，这两个数比盯着矩阵本身更有用。

19.34 第五步：如果还要转回三维 ENU 怎么做

RHI 只是固定在一个方位角上的二维切片。如果还要把剖面质心放回三维地图，则需要再把固定方位角 40 deg 代入：

$$\begin{aligned} x_c &= s_c \sin(40^\circ) \\ y_c &= s_c \cos(40^\circ) \\ z_c &= h_c \end{aligned}$$

得到近似三维质心：

$$(x_c, y_c, z_c) \approx (83.9, 100.0, 38.7) \text{ m}$$

和前面的 PPI 热点相比，你会发现：

1. 水平位置仍然接近同一片区域。
2. 但这次额外知道了它在竖直方向上的主中心高度更高。
3. 这就是为什么 PPI 和 RHI 最好配合使用，而不是只用其中一种。

19.35 第六步：如果把这个 RHI 结果画到 UI 上，会长什么样

在 RHI 页面上，它更像下面这种“地距-高度”结构：

```
RHI fixed azimuth = 40 deg

elevation \ range    60m    90m    120m   150m
4 deg          low     low     high    high
8 deg          low     low     high    high
12 deg         low     low     high    high
16 deg         low     low     high    low

section centroid -> ground_range=130.5 m, height=38.7 m
layer bottom    -> 26.4 m
layer top       -> 51.1 m
```

前端真正展示时，通常会把它画成：

1. 距离-高度热力图。
2. 叠加热点轮廓线。
3. 叠加层顶、层底和质心标记。
4. 必要时叠加风向或喷雾可达范围。

19.36 第七步：用代码把这个 RHI 算例串起来

下面是一段示意代码，把二维 RHI 小矩阵从阈值筛选一路串到剖面质心：

```
import numpy as np

elevations = np.array([4.0, 8.0, 12.0, 16.0], dtype=float)
ranges = np.array([60.0, 90.0, 120.0, 150.0], dtype=float)
sensor_height = 18.0

pm25 = np.array([
    [28.0, 45.0, 72.0, 65.0],
    [32.0, 58.0, 95.0, 84.0],
    [30.0, 55.0, 88.0, 76.0],
    [24.0, 41.0, 62.0, 54.0],
])

mask = pm25 > 60.0
section_points = []

for iel, elevation in enumerate(elevations):
    el = np.deg2rad(elevation)

    for ir, radius in enumerate(ranges):
        if not mask[iel, ir]:
            continue

        value = pm25[iel, ir]
        ground_range = radius * np.cos(el)
        height = sensor_height + radius * np.sin(el)
        section_points.append((value, ground_range, height))

weights = np.array([row[0] for row in section_points])
sh = np.array([[row[1], row[2]] for row in section_points])
centroid = (weights[:, None] * sh).sum(axis=0) / weights.sum()

layer_bottom = sh[:, 1].min()
layer_top = sh[:, 1].max()

print("section_point_count =", len(section_points))
print("section_centroid_sh =", np.round(centroid, 2))
print("layer_bottom_top =", np.round([layer_bottom, layer_top], 2))
```

这段代码的逻辑很直接：

1. 先在二维 RHI 矩阵上做阈值筛选。
2. 再把超阈值格子转成剖面坐标。
3. 再从这些剖面点里提取层底、层顶和质心。

19.37 这个 RHI 算例真正想让你学会什么

这一节真正想让你建立的是下面这套判断框架：

1. PPI 更擅长看水平扩散和热点平面位置。
 2. RHI 更擅长看垂直结构、抬升高度和层顶层底。
 3. 如果只看 PPI，你知道热点在哪，但不一定知道它抬多高。
 4. 如果只看 RHI，你知道它抬多高，但不一定知道它横向扩到哪。
 5. 真正成熟的业务系统，往往需要 PPI 和 RHI 配合，才能同时支持告警、研判和联动控制。
-

20. 一个适合你的学习顺序

如果你现在真的是零基础，建议就按这个顺序学，不要跳：

1. 先看懂“激光打一枪，为什么能换成距离”。
2. 再看懂 LiDAR 方程每一项的物理意思。
3. 再理解为什么不能直接从回波读出 PM 浓度。
4. 再学 RCS、Klett/Fernald、湿度修正这几个实用算法。
5. 再看 ENU 坐标和 2D/3D UI。
6. 最后才去碰多波长、偏振、Raman、DIAL 这些更高级内容。

只要前 4 步走扎实，你对这套系统的理解就已经不再是“听起来很高级”，而是真正能跟工程团队讨论需求了。

21. 参考资料

下面这些资料适合继续往下读：

1. [NOAA LIDAR principle page](#)
2. [JPL NDACC instruments overview](#)
3. [CloudnetPy quickstart](#)
4. [Klett 1981 paper abstract](#)
5. [Fernald 1984 paper abstract](#)
6. [PyMieScatt documentation](#)
7. [Gaudfrin et al. 2020, AMT article with inversion flow and signal figures](#)
8. [CloudCompare presentation page stating the UI relies on Qt and OpenGL](#)
9. [QGIS screenshot: application for topological wastewater management](#)
10. [QGIS screenshot: Trajectools dark mode](#)

如果后面你要继续深化，我建议下一份文档再单独拆成三部分：

1. 纯入门版，只讲概念和图。
2. 算法版，只讲反演和标定。
3. 工程版，只讲设备、通信、UI 和联动控制。